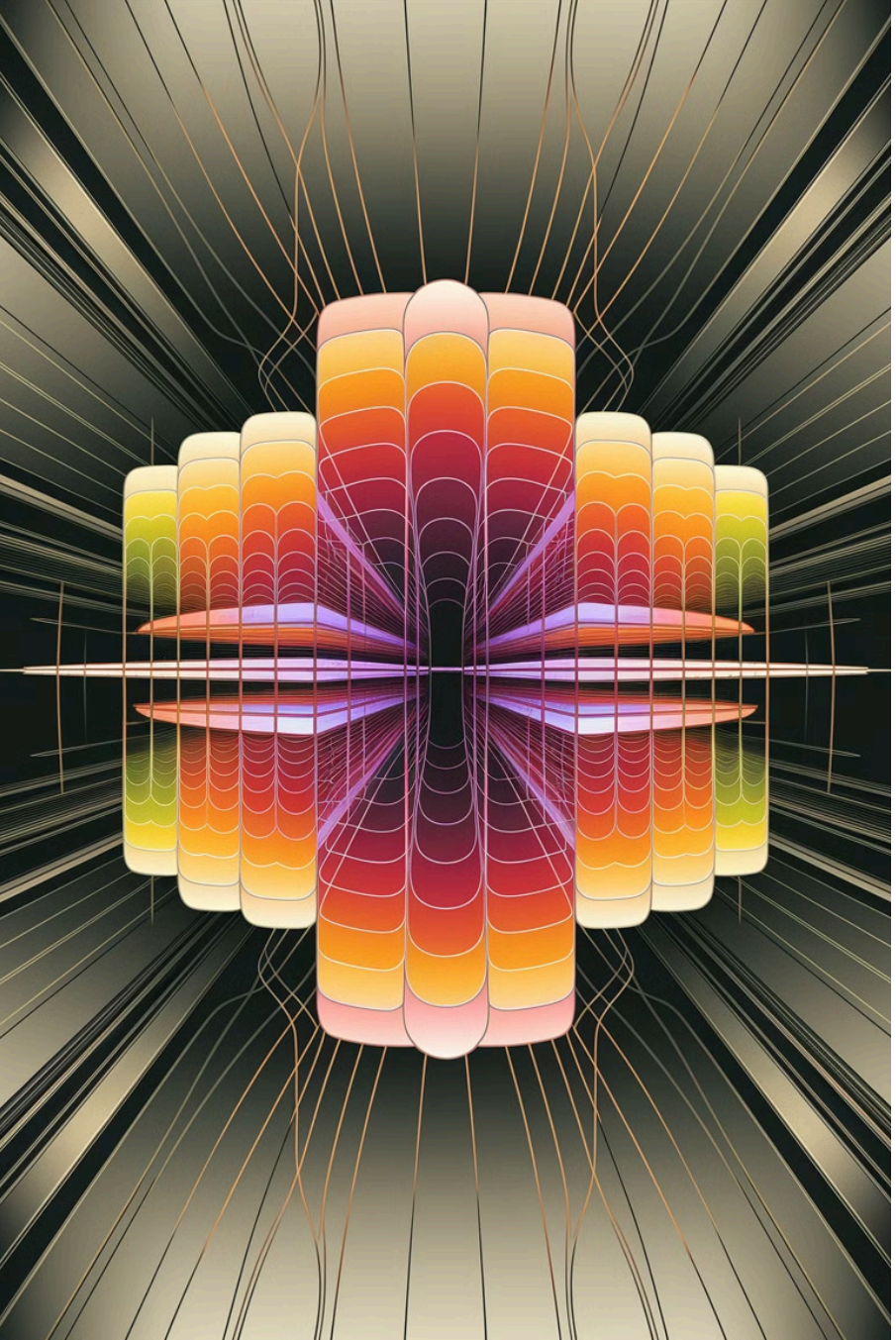


Lecture 5

Transformers for Language Modeling
The Power of Self-Attention

MAI554: Deep Learning for NLP

Prof. Anis Koubaa

Professor

College of Engineering
Alfaisal University

akoubaa@alfaisal.edu

Recap & Motivation

1. **FFN**

   Basic feedforward networks

2. **RNN**

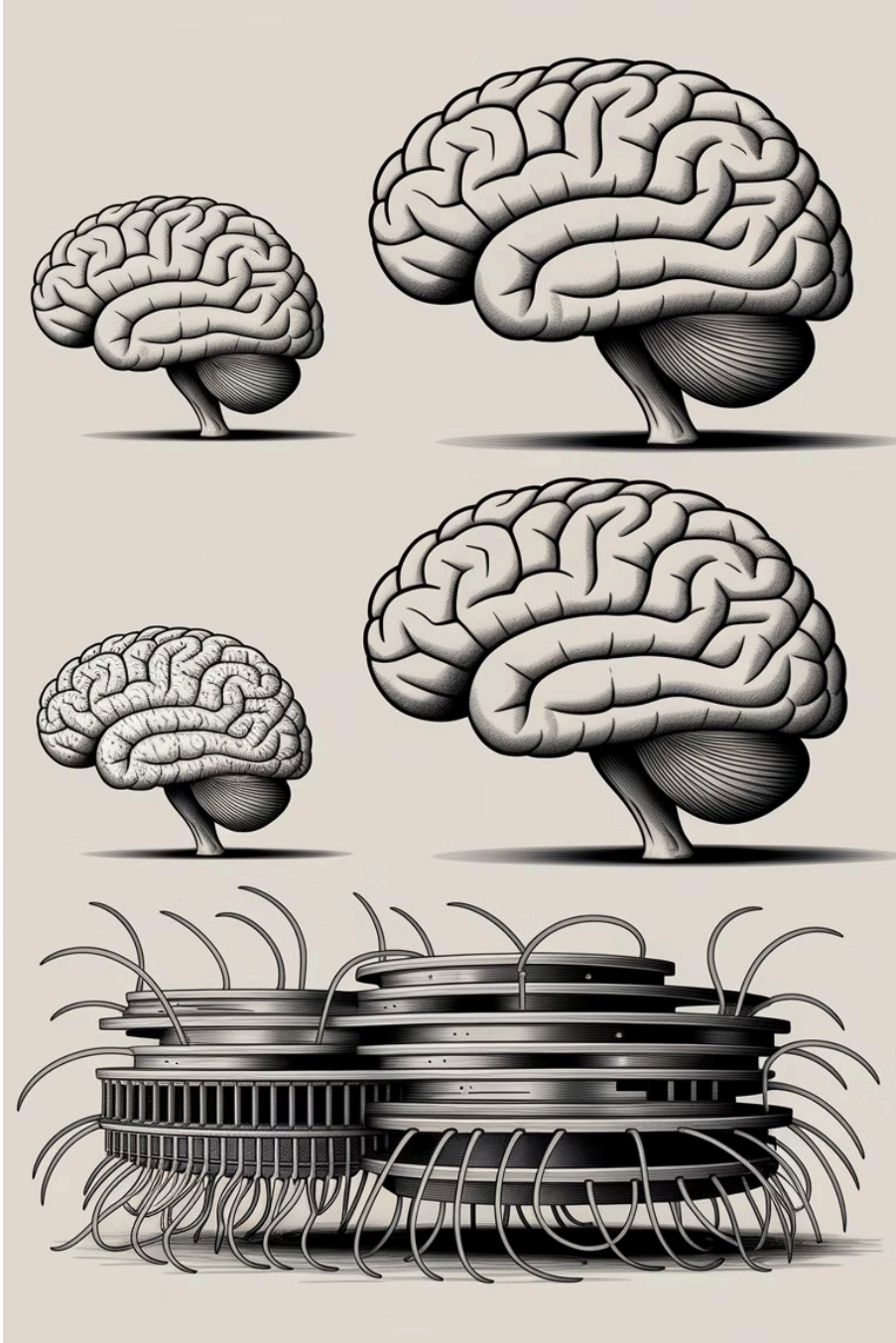
   Recurrent Neural Networks for sequential data

3. **LSTM**

   Long Short-Term Memory for better long-range dependencies

4. **Transformer**

   Parallel processing and improved efficiency



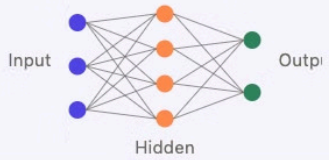
Language Modeling Evolution

Language Model Evolution

Feed-Forward Networks

Fixed-size input windows predicting next tokens based on local context.

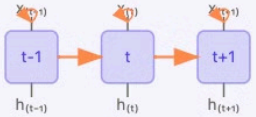
No sequential dependencies, poor long-context understanding .



Recurrent Neural Networks

Hidden states carry context across time steps, processing variable-length sequences .

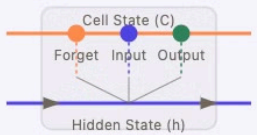
Vanishing gradients limit long-range dependencies.



LSTM Networks

Gating mechanisms regulate information flow, reducing vanishing gradients .

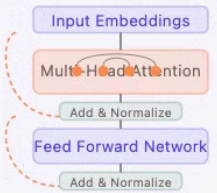
Sequential processing limits parallelization and scalability.



Transformers

Self-attention enables parallel processing of entire sequences.

Quadratic complexity in sequence length. Enables scalable translation and text generation.



Why Transformers

Why Transformers?

Key advantages of Transformer architectures over RNNs and LSTMs

1. Parallel Processing

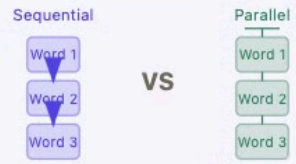
RNNs/LSTMs

Process tokens sequentially, making training slow for long sequences.

vs

Transformers

Process all tokens simultaneously with self-attention, enabling faster training.



2. Long-Range Connections

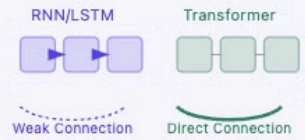
RNNs/LSTMs

Struggle with long sequences. Distant token relationships are difficult to capture.

vs

Transformers

Direct connections between all tokens, making long-range dependencies easier to model.



3. Context Handling

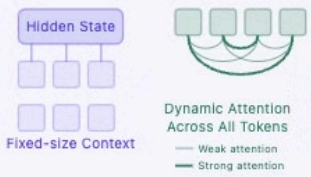
RNNs/LSTMs

Use fixed-length hidden states to summarize past information, limiting context understanding.

vs

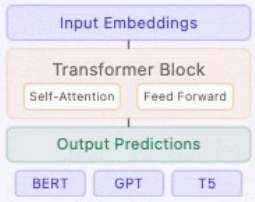
Transformers

Dynamic attention across the entire sequence, weighting tokens based on relevance.



Impact

Transformers enabled breakthrough models like BERT, GPT, and T5, revolutionizing NLP capabilities.



Why Transformers II

Key limitations of RNNs/LSTMs that led to Transformer architectures

1. Memory Bottlenecks

Memory scaling with sequence length:

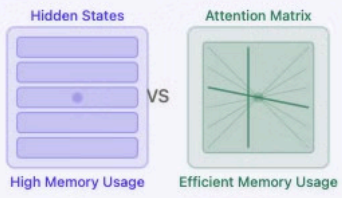
⊕ RNNs/LSTMs

Hidden states for each time step create high memory usage, making training infeasible for long sequences.

vs

⊖ Transformers

Self-attention scales more efficiently, allowing larger batch sizes and longer sequences.



2. Complex Relationships

Modeling non-sequential dependencies:

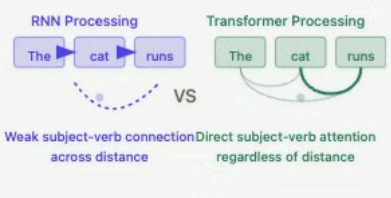
⊕ RNNs/LSTMs

Sequential processing struggles with complex relationships like subject-verb agreement across long distances.

vs

⊖ Transformers

Direct pairwise attention between all tokens makes modeling complex linguistic dependencies easier.



3. Exposure Bias

Training vs. inference behavior:

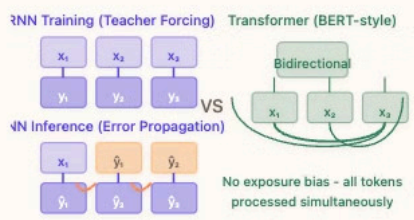
⊕ RNNs/LSTMs

Teacher forcing during training vs. using own predictions at inference leads to error accumulation.

vs

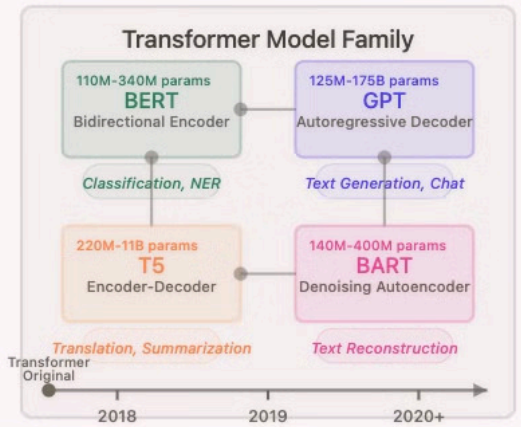
⊖ Transformers

Parallel processing in models like BERT reduces exposure bias by handling entire sequences simultaneously.

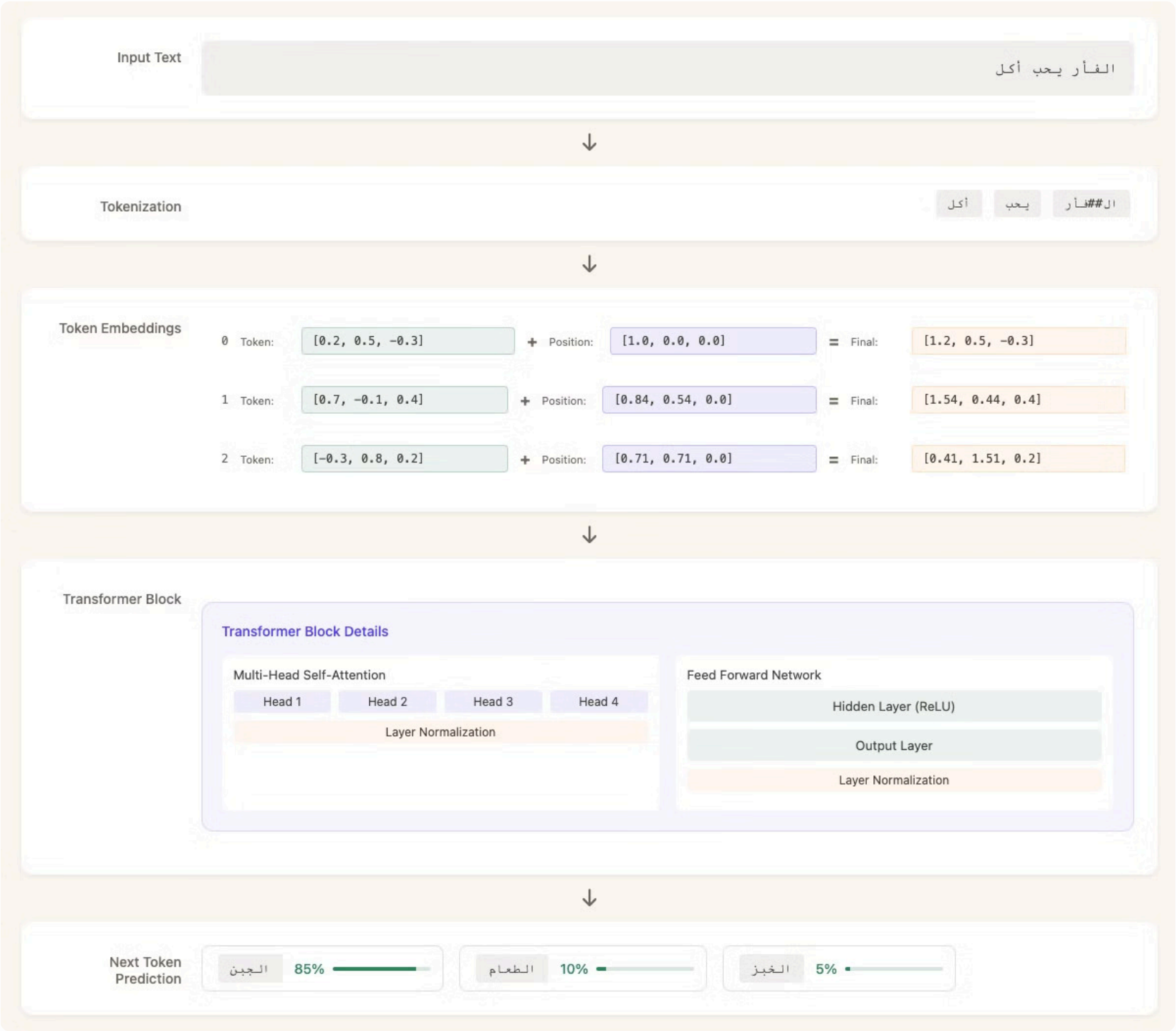


Transformer Evolution

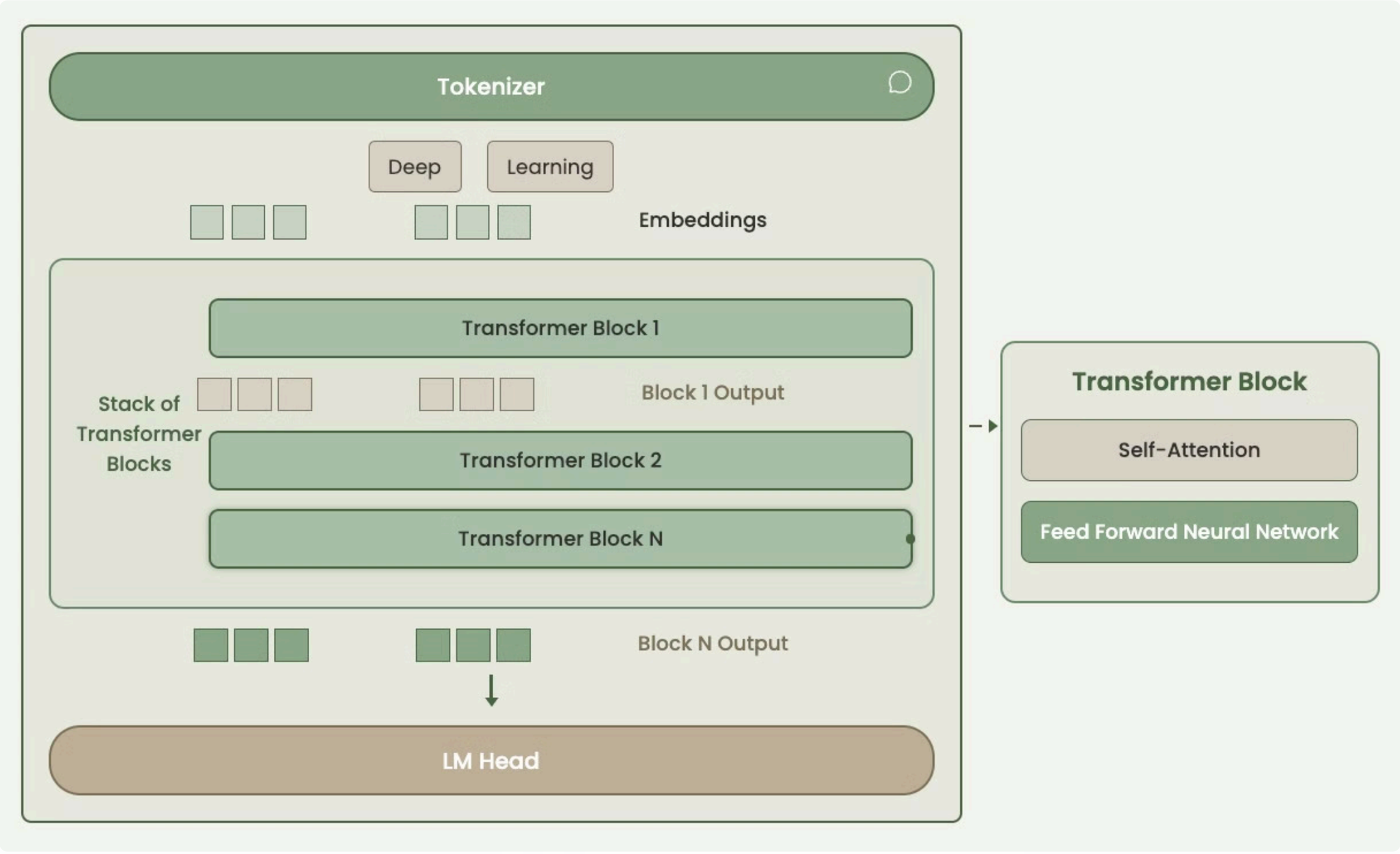
Key transformer variants and their applications:



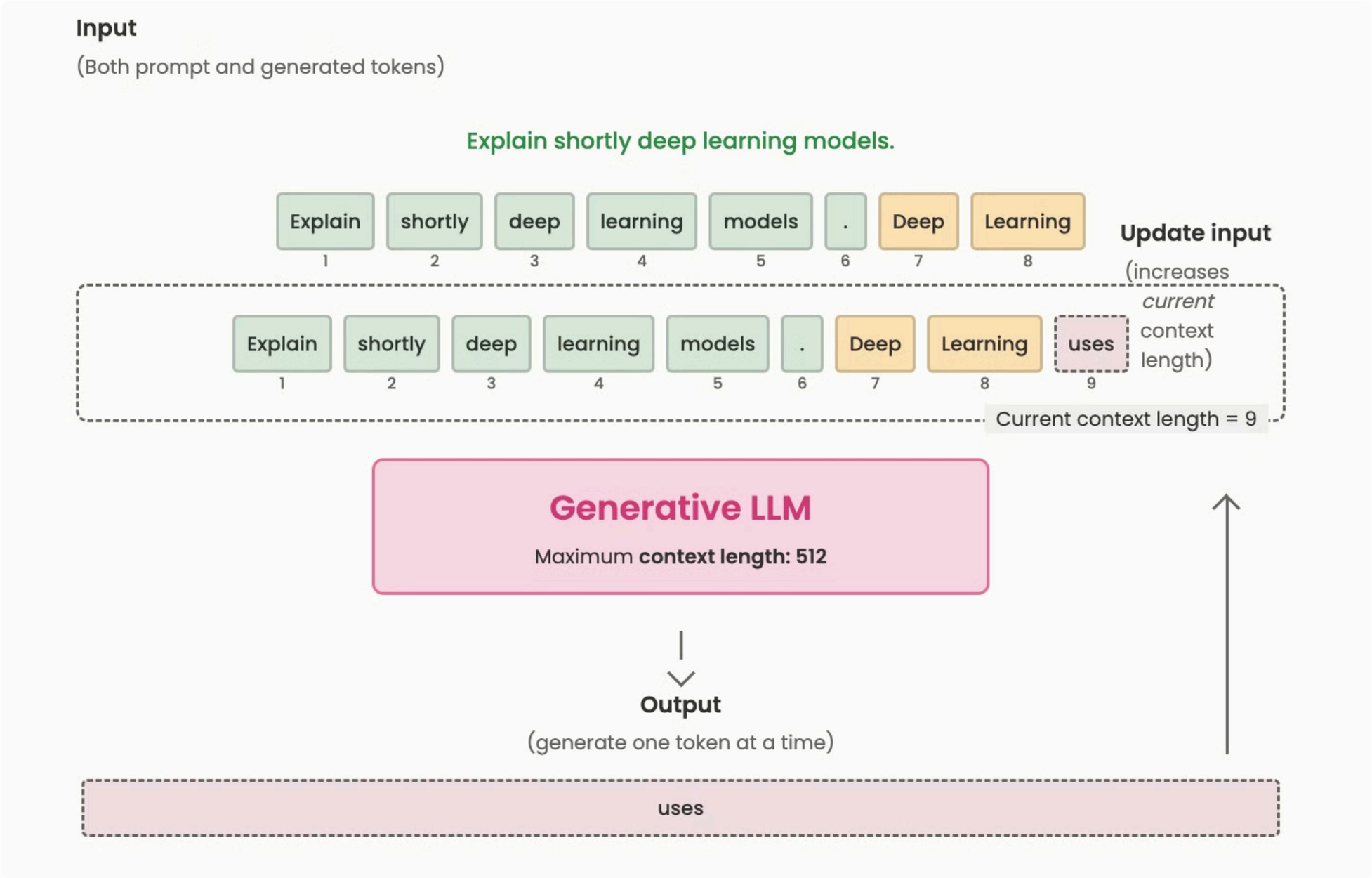
Transformer Architecture



Transformer Block



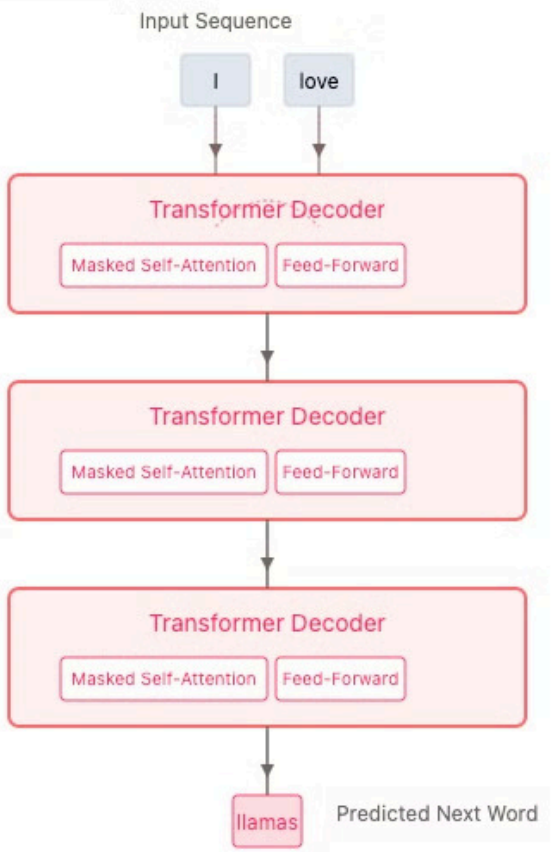
Token Generation Process



Transformer Architecture Comparison

Generative Models (GPT-1)

Decoder-Only Architecture

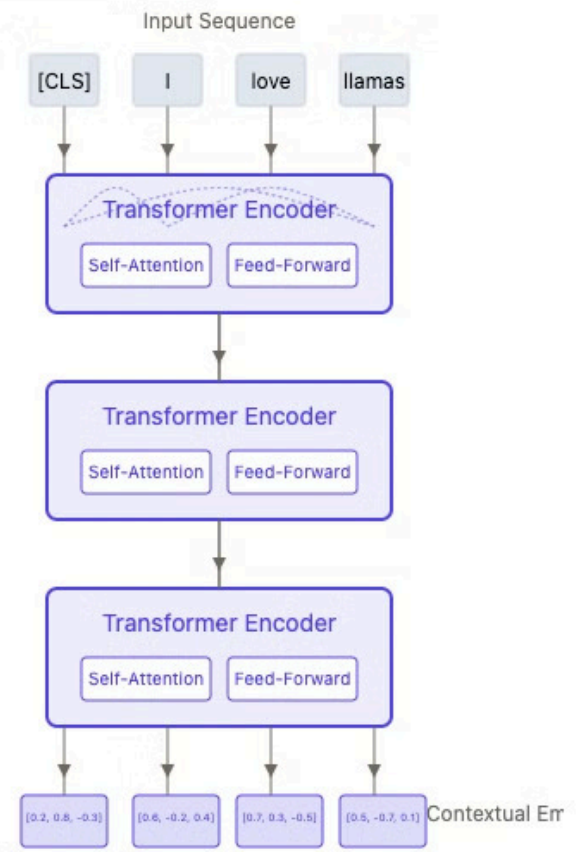


- **Decoder-Only Architecture** - Uses only transformer decoder layers
- **Masked Self-Attention** - Can only see previous tokens, not future ones
- **Autoregressive** - Outputs one token at a time, using previous outputs as inputs
- **Left-to-Right Processing** - Processes text in a single direction

Example Tasks: *Text generation, completion, summarization, and translation*

Representation Models (BERT)

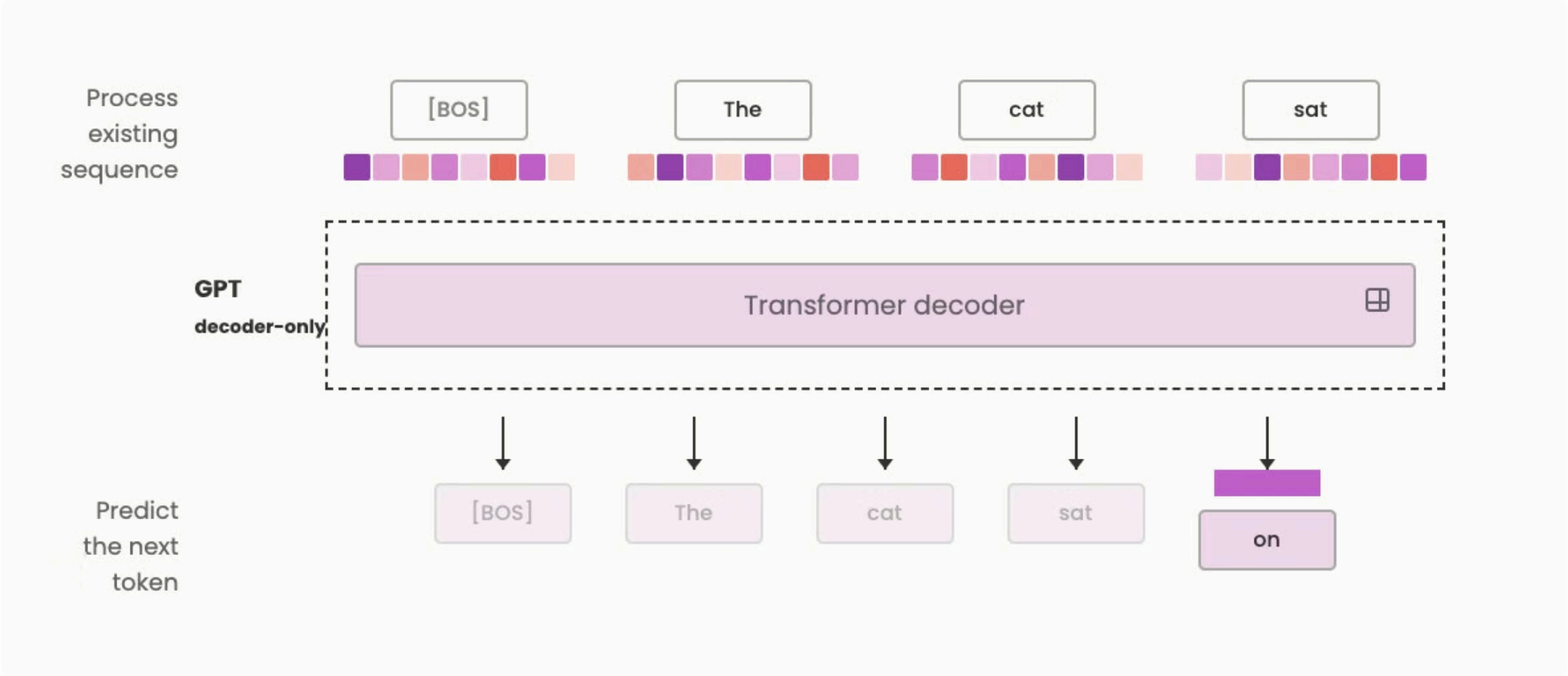
Encoder-Only Architecture



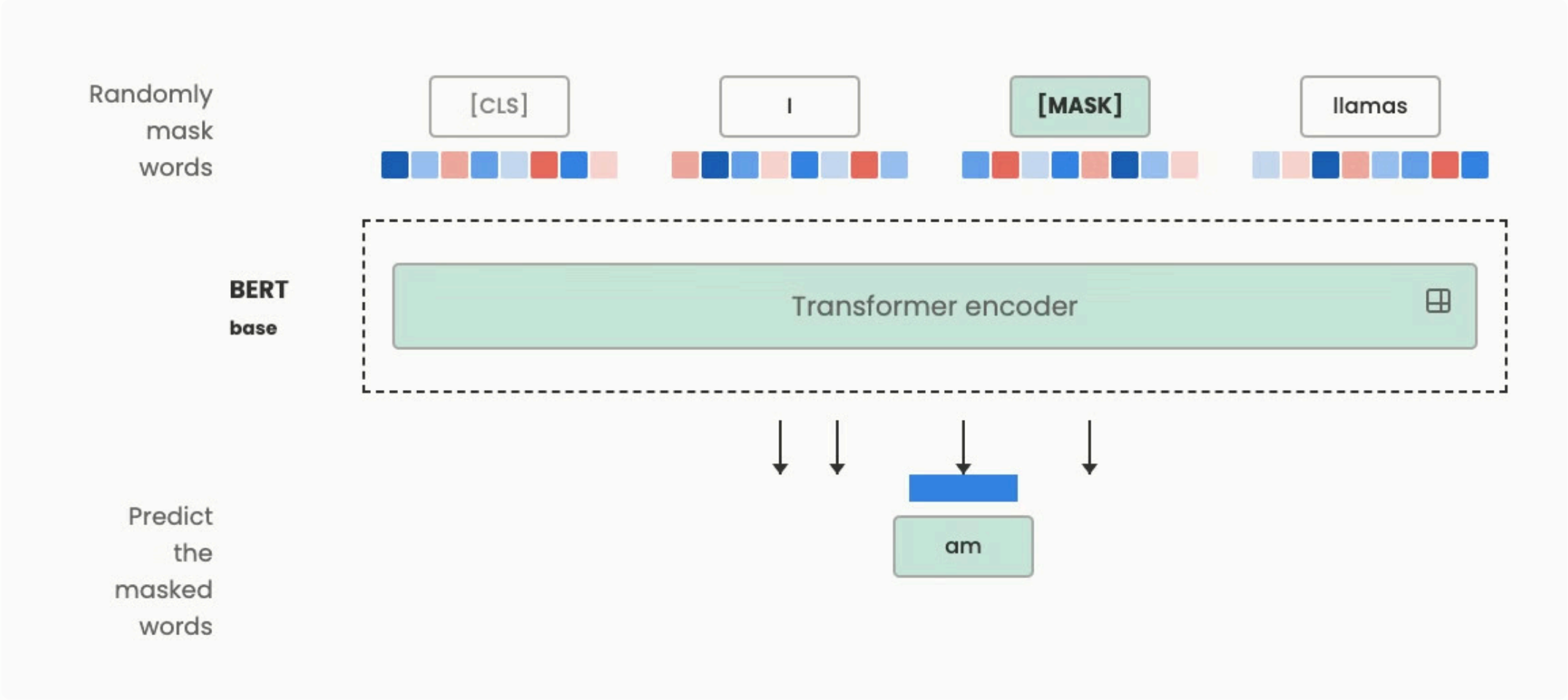
- **Encoder-Only Architecture** - Uses only transformer encoder layers
- **Bidirectional Self-Attention** - Can see all tokens in the sequence simultaneously
- **Contextual Representations** - Outputs embeddings for all tokens in the sequence
- **Special Tokens** - Uses [CLS] token for classification tasks and [SEP] for sequence separation

Example Tasks: *Sentence classification, named entity recognition, question answering, and token classification*

Generative Models



Representational Models



How to Encode Position with Transformers?

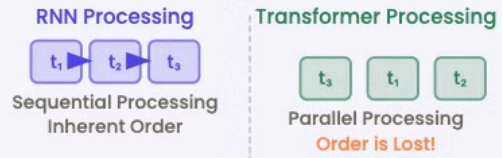
The Need for Positional Encoding in Transformers

The Need for Positional Encoding in Transformers

Why Transformers require explicit position information and how it's implemented

The Problem: Order Agnosticism

Unlike RNNs, Transformers process all tokens **simultaneously**. This makes them **order-agnostic** - they don't inherently know the position of each token in a sequence.



Example sentences with same tokens but different meanings:

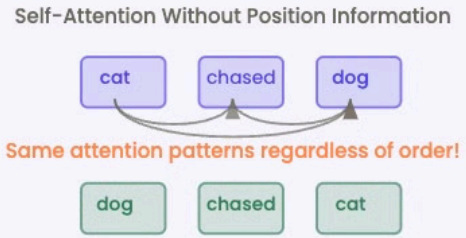
"The cat chased the dog" ≠ "The dog chased the cat"

Why Order Matters

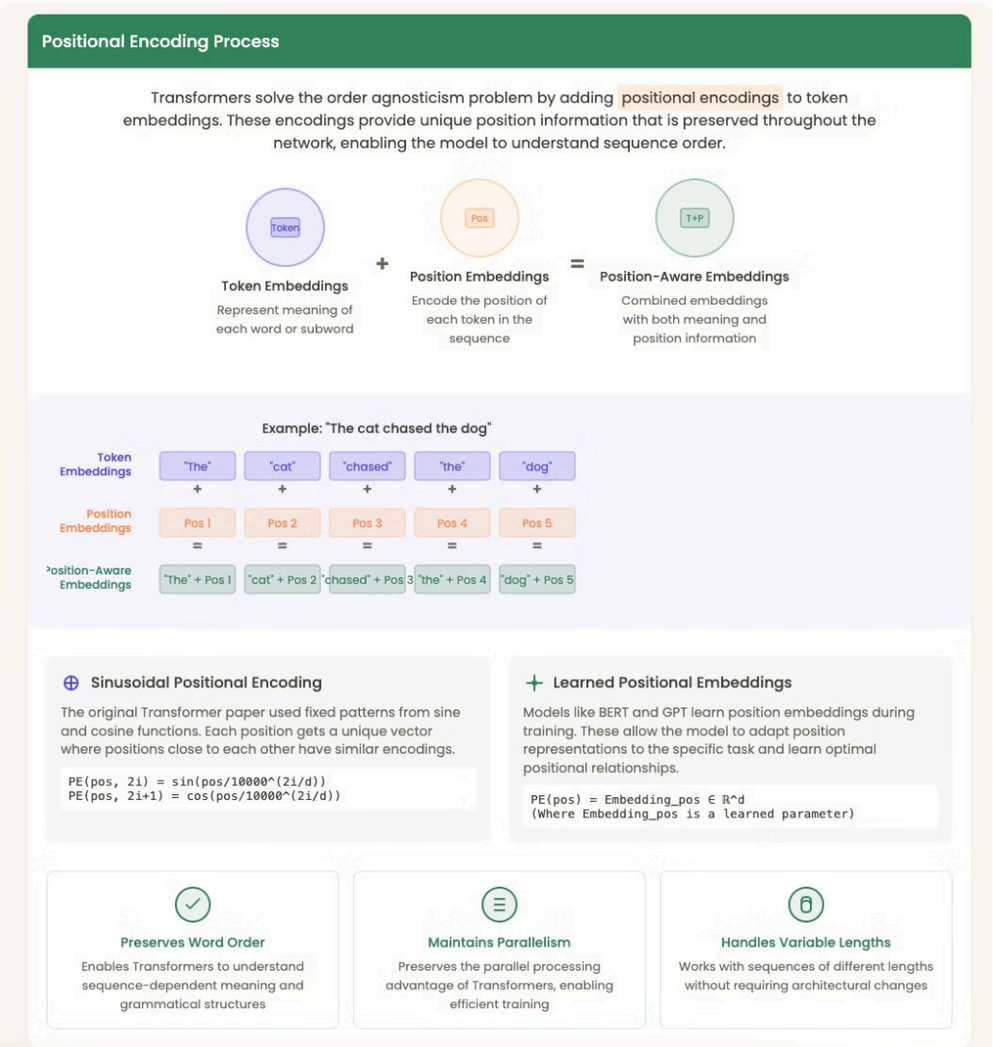
In language, **word order** is crucial. The same words in different orders can have **completely different meanings**.

Without Position Information:

Transformers would treat "The cat chased the dog" and "The dog chased the cat" as **identical**, since attention only computes token relationships regardless of position.



Introduction to Positional Encoding



Why Do We Need Positional Encoding?

- Transformers process all words **simultaneously** instead of sequentially (unlike RNNs/LSTMs).
- They **lack an inherent sense of word order**, which is crucial for understanding meaning.
- **Solution:** Add **positional encoding** to word embeddings to provide ordering information.

How Does Positional Encoding Work?

- Each word in the sentence (e.g., "He wants to play football with friends") is first converted into a **word embedding** (🔹 blue boxes in the image).
- A **positional encoding** (🟡 yellow boxes) is generated for each word based on its position in the sentence.
- The **word embedding and positional encoding are summed** (⊕ operation in the image).
- This produces **position-aware embeddings** (🟩 green boxes), which retain both **word meaning** and **order information**.

SS

Positional Encoding Process

Transformers solve the order agnosticism problem by adding positional encodings to token embeddings. These encodings provide unique position information that is preserved throughout the network, enabling the model to understand sequence order.

Token Embeddings

Represent meaning of each word or subword

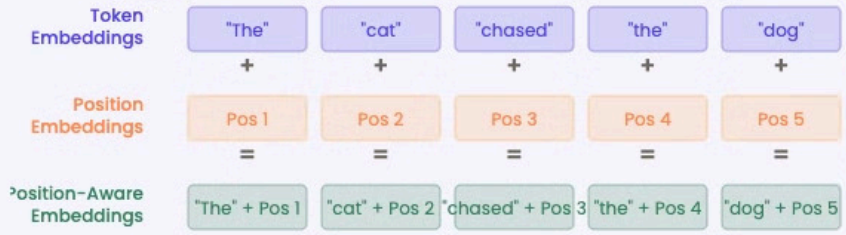
Position Embeddings

Encode the position of each token in the sequence

Position-Aware Embeddings

Combined embeddings with both meaning and position information

Example: "The cat chased the dog"



Sinusoidal Positional Encoding

The original Transformer paper used fixed patterns from sine and cosine functions. Each position gets a unique vector where positions close to each other have similar encodings.

```
PE(pos, 2i) = sin(pos/10000^(2i/d))
PE(pos, 2i+1) = cos(pos/10000^(2i/d))
```

Learned Positional Embeddings

Models like BERT and GPT learn position embeddings during training. These allow the model to adapt position representations to the specific task and learn optimal positional relationships.

```
PE(pos) = Embedding_pos ∈ ℝ^d
(Where Embedding_pos is a learned parameter)
```

Preserves Word Order

Enables Transformers to understand sequence-dependent meaning and grammatical structures

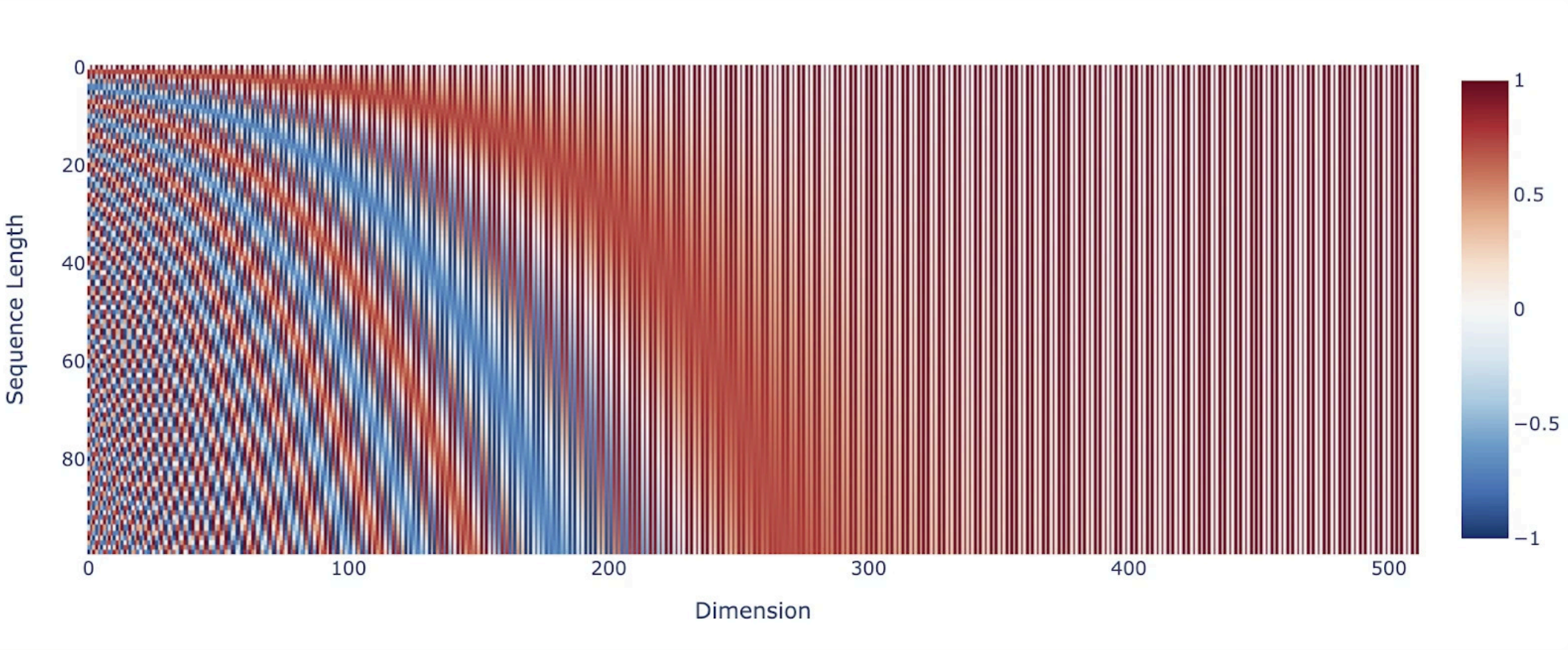
Maintains Parallelism

Preserves the parallel processing advantage of Transformers, enabling efficient training

Handles Variable Lengths

Works with sequences of different lengths without requiring architectural changes

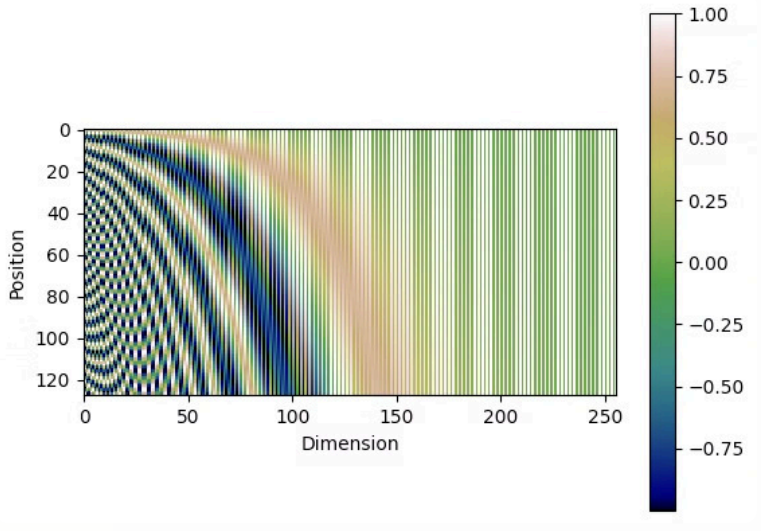
Sinusoidal Position Encoding



Sinusoidal Position Encoding

position.py

```python
def position_encoding(
    seq_len: int, dim_model: int, device: torch.device = torch.device("cpu"),
) -> Tensor:
    pos = torch.arange(seq_len,
                       dtype=torch.float,
                       device=device).reshape(1, -1, 1)

    dim = torch.arange(dim_model,
                       dtype=torch.float,
                       device=device).reshape(1, 1, -1)

    phase = pos / (1e4 ** (dim / dim_model))

    return torch.where(dim.long() % 2 == 0, torch.sin(phase), torch.cos(phase))
```



Sinusoidal Position Encoding

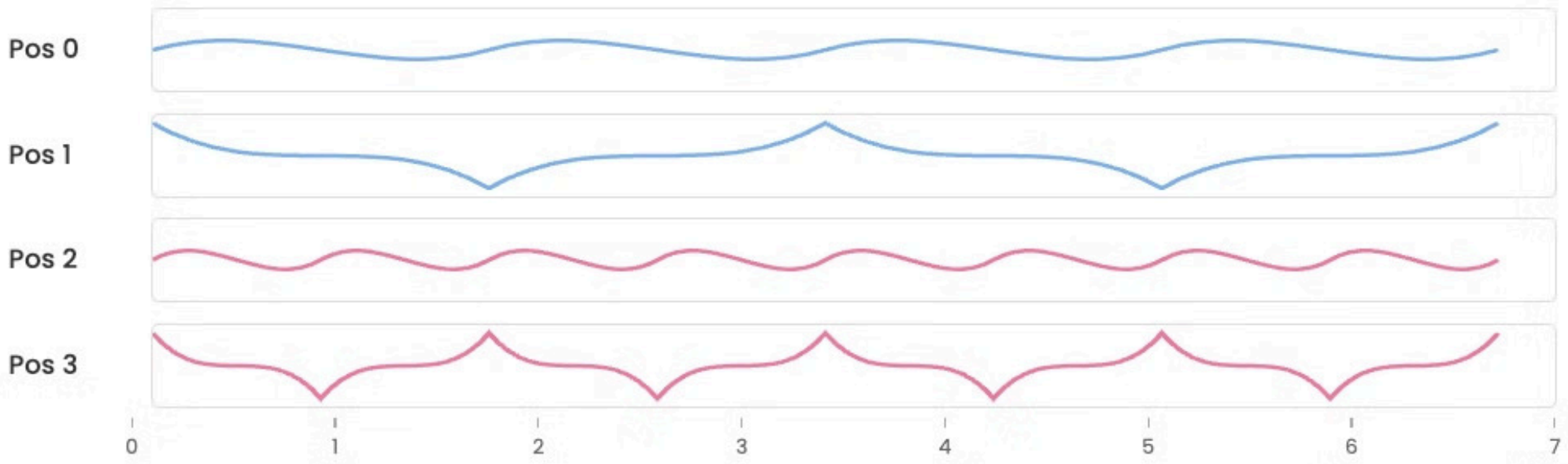
Unlike BERT's learned embeddings, **sinusoidal encoding** uses **sine and cosine** functions to encode token positions. These encodings are added to word embeddings before processing.

Formula

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{2i/d})$$

pos = word position ***i*** = dimension index ***d*** = embedding dimension



Key Advantages

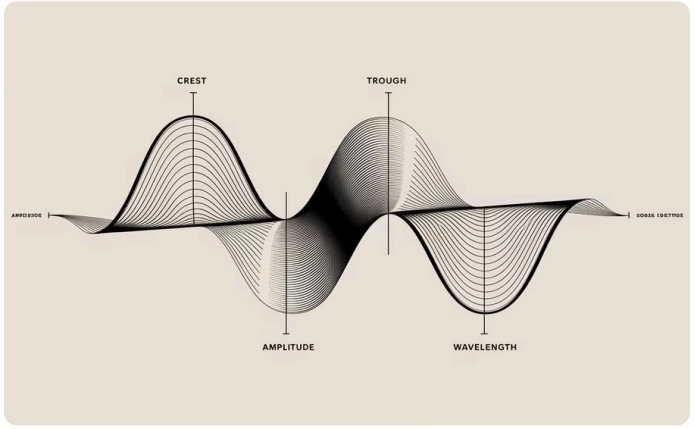
Unique Position Encoding
Each position gets a distinct encoding vector.

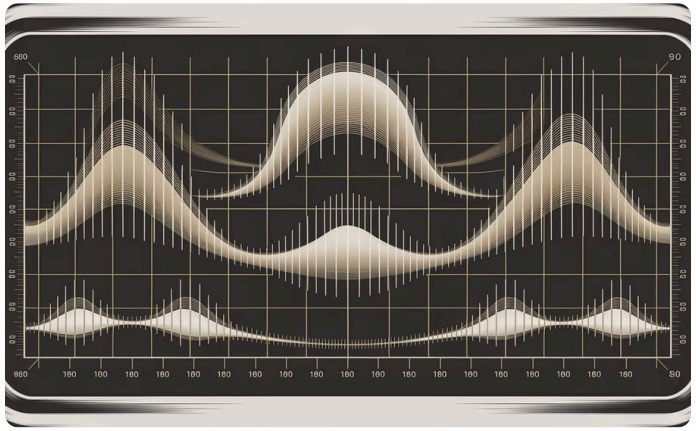
Structured Patterns
Wave-like structures help the model learn relative positions.

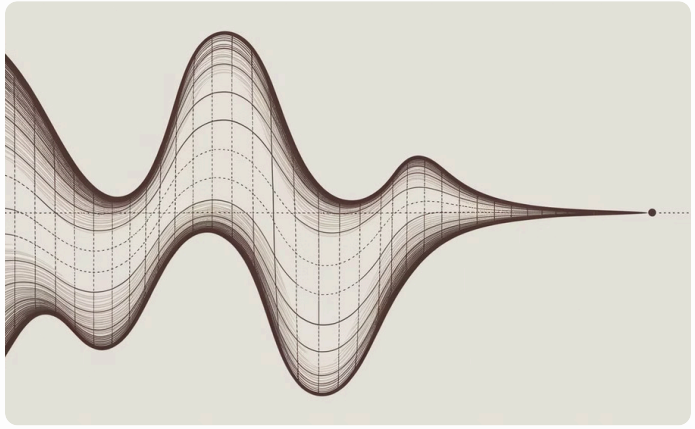
Length Generalization
Works with sequences longer than seen during training.

Sinusoidal encoding enables Transformers to understand word order while maintaining parallel processing efficiency.

Why Sine and Cosine?







Unique for Each Position

Every word receives a unique encoding, enabling the model to distinguish between different positions in the sequence.

Captures Patterns

The functions create repeating wave-like structures, making it easier for the model to learn relative positions.

Allows Extrapolation

Unlike learned embeddings, sinusoidal encoding works even for sequences longer than those seen during training.

Pros and Cons of Positional Encoding

Pros

- **Emphasis on Relative Position**

  The cyclical pattern in sinusoidal encoding highlights the importance of relative positions in sequences, making it easier for the model to understand relationships between tokens that are at similar distances from each other.

- **Extension to Longer Sequences**

  Its repetitious nature can potentially facilitate encoding longer sequences than those seen during training, allowing models to generalize to variable-length inputs.

- **No Additional Parameters**

  Uses a deterministic function rather than learned parameters, keeping the model size smaller and training more efficient.

Cons

- **Non-learnable Encoding**

  Sinusoidal encoding is fixed and doesn't adjust based on the task or data during training, potentially limiting how well it can capture position information for specific domains.

- **Potential Inaccuracy for Extended Sequences**

  Repetitive encoding might lead to unreliable representations in extended sequences, where positions far beyond the training distribution need to be represented.

- **Fixed Frequency Spectrum**

  The predetermined set of frequencies may not be optimal for all sequence lengths or types of data, potentially limiting performance in specialized tasks.

Positional Encoding vs. Positional Embedding

Feature	**Positional Encoding** (Sinusoidal)	**Positional Embedding** (Learned)
Definition	Uses a **fixed mathematical function** (sine & cosine) to encode positions.	**Learns** position embeddings during training like word embeddings.
Generalization	Can generalize to **longer sequences** not seen in training.	Limited to positions seen during training; struggles with longer sequences.
Computation	**No learning required**; directly computed using a formula.	Requires **trainable parameters**, increasing model complexity.
Interpretability	Easier to interpret due to its **structured periodic nature**.	Less interpretable, since embeddings are **learned** rather than defined.
Memory Efficiency	Requires **no additional trainable parameters**, making it more memory-efficient.	Uses additional **trainable parameters**, increasing memory usage.
Example Usage	**Transformers** (original Vaswani et al., 2017).	**BERT, GPT, T5** (modern transformers with learned embeddings).

Learnable Positional Embeddings

📌 What Are They?

- Instead of using a fixed formula, the model **learns position embeddings** during training.
- Each position gets a **trainable vector**, just like word embeddings.

🛠️ How Do They Work?

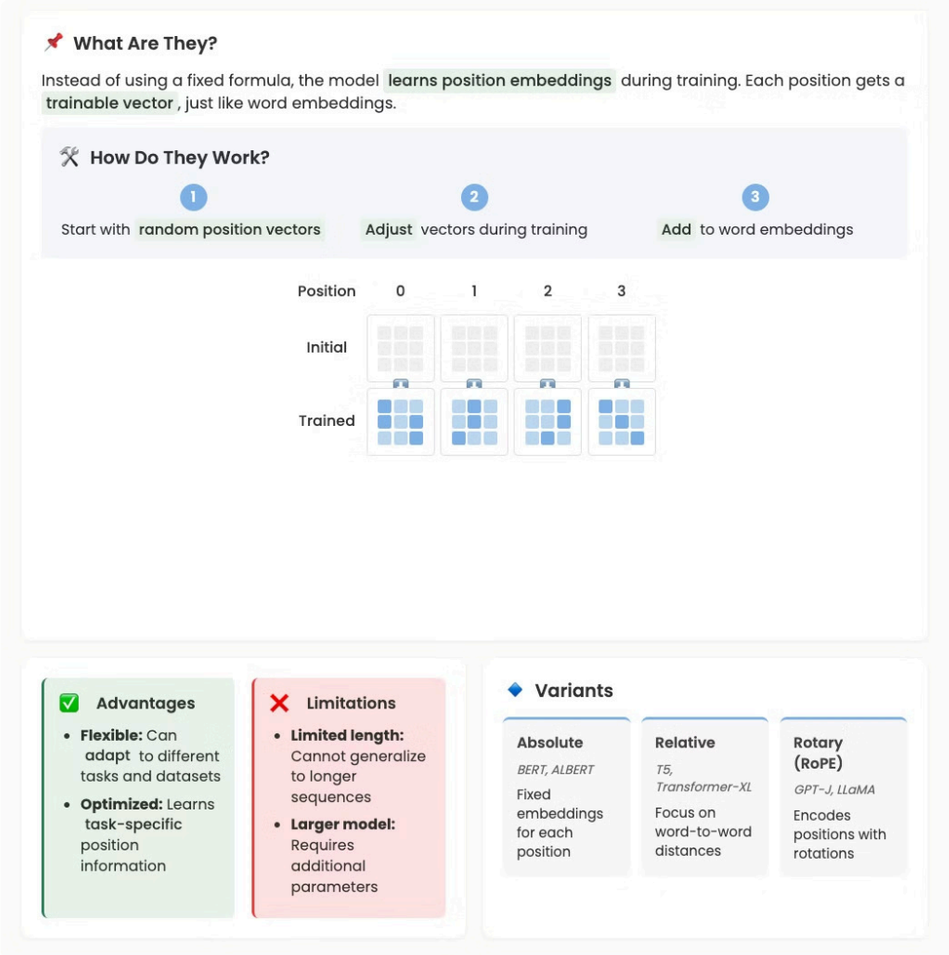
- The model starts with **random position vectors**.
- During training, these vectors are **adjusted** to capture useful position information.
- They are **added to word embeddings** before passing into the Transformer.

✅ Advantages (Pros)

- **Flexible:** Can **adapt** to different tasks and datasets.
- **Optimized for performance:** The model learns **what position information is most useful**.

❌ Limitations (Cons)

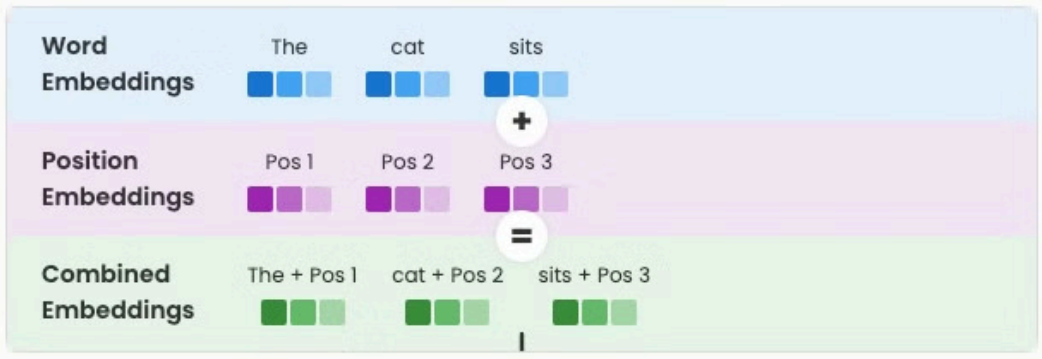
- **Cannot generalize to longer sequences** than seen in training.
- **Requires additional parameters**, increasing model size.



Learnable Embedding

📌 What Are They?

- Instead of using a fixed formula, the model **learns position embeddings** during training.
- Each position gets a **trainable vector**, just like word embeddings.



🛠️ How Do They Work?

- The model starts with **random position vectors**.
- During training, these vectors are **adjusted** to capture useful position information.
- They are **added to word embeddings** before passing into the Transformer.

✅ Advantages (Pros)

- **Flexible:** Can **adapt** to different tasks and datasets.
- **Optimized for performance:** The model learns **what position information is most useful**.

❌ Limitations (Cons)

- **Cannot generalize to longer sequences** than seen in training.
- **Requires additional parameters**, increasing model size.

Positional Embedding Approaches

Absolute Positional Embeddings (APE)

- APE assigns a **unique**, **learnable vector** to each **position** in a **sequence**.
- This method is **simple** and **effective**, particularly for tasks where **absolute word order** is crucial.

Relative Positional Embeddings (RPE)

- RPE focuses on the **relative distances** between words rather than **absolute positions**.
- It allows the model to handle **variable-length sequences** and excels in tasks requiring **reasoning** and understanding of **contextual relationships**.

Rotary Positional Embeddings (RoPE)

- RoPE utilizes **rotational transformations** to encode both **absolute** and **relative positions** within token representations.
- This approach offers **scalability**, **generalization**, and a balance of **absolute** and **relative positioning**, making it highly effective for **large language models**.

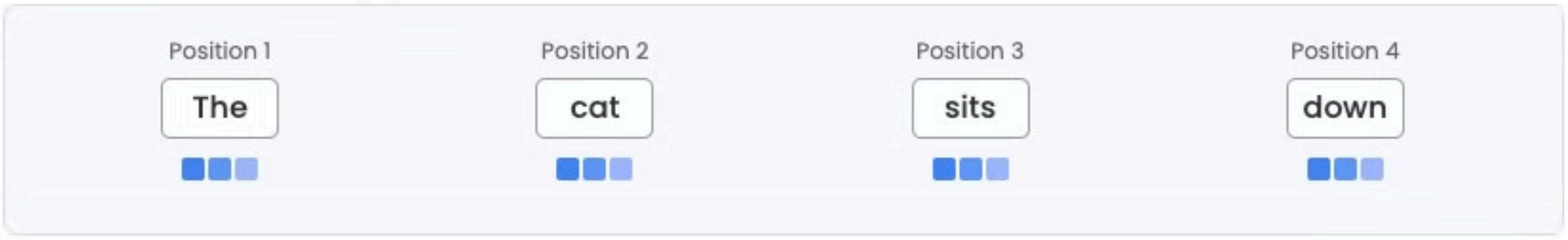
Absolute Positional Embeddings (APE)

1 Absolute Positional Embeddings (APE)

Used in BERT, ALBERT

How It Works

- Each position in a sequence (1, 2, 3, ...) is assigned a **learnable vector** during training.
- These vectors are **fixed** after training, meaning the same position always gets the same encoding.
- The embeddings are **added** to word embeddings before processing by the Transformer.



✅ Pros

- **Simple and effective** – Easy to implement and works well for NLP tasks.
- **Captures absolute word positions** – Helps in tasks where order matters (e.g., sentence classification).

❌ Cons

- **Cannot generalize to longer sequences** – If a model is trained on 512-token sequences, it struggles with longer ones.
- **Ignores relative relationships** – It does not explicitly encode how words relate to each other in position.

Relative Positional Embeddings (RPE)

2 Relative Positional Embeddings (RPE)

Used in T5, Transformer-XL

How It Works

- Instead of assigning a unique vector to each position, RPE **computes distances between words**.
- The model **learns relative positions**, like "word A is 3 positions ahead of word B."
- These relative distances help capture **contextual relationships** in sequences.

✅ Pros

- **Handles long texts better** – Can process variable-length sequences without fixed limits.
- **Better for reasoning and structured tasks** – Since it understands **how** words relate rather than just their absolute positions.

❌ Cons

- **More complex to implement** – Since the model must compute and update distances dynamically.
- **Might not be ideal for all tasks** – Some tasks benefit from absolute positions, like chronological order tracking.

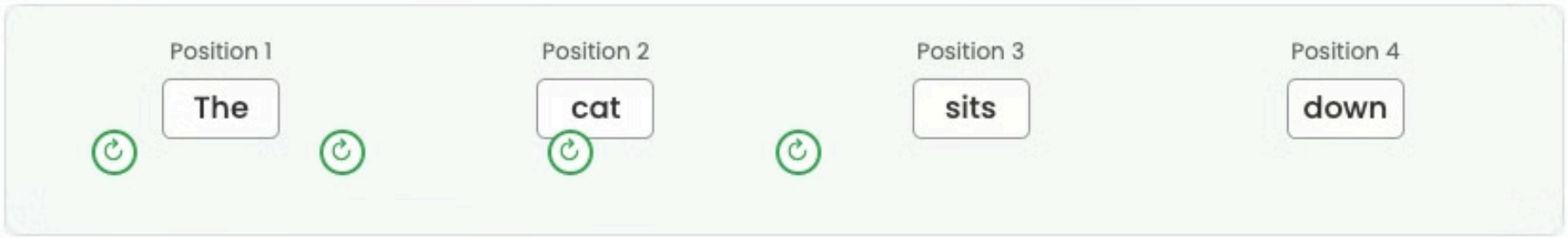
Rotary Positional Embeddings (RoPE)

3 Rotary Positional Embeddings (RoPE)

Used in GPT-4, Llama, and some newer models

How It Works

- Uses **rotational transformations** to encode positional information **directly into token representations**.
- This allows the model to **capture both absolute and relative positions** at the same time.
- RoPE applies **complex number rotations** to embeddings, creating a continuous representation of positions.



✅ Pros

- **Best of both worlds** – Captures **absolute and relative** positioning effectively.
- **Scales well to longer sequences** – Works even with texts **longer than training data**.
- **Improves generalization** – Helps in **few-shot learning** and large-scale language models.

❌ Cons

- **More computational overhead** – The rotational transformations require extra processing.
- **Requires tuning for best performance** – Unlike simple APE, RoPE may need **hyperparameter adjustments**.

Key Differences

🔍 Key Differences & Summary

Feature	Absolute (APE) BERT, ALBERT	Relative (RPE) T5, Transformer-XL	Rotary (RoPE) GPT-4, LLaMA
Positional Reference	Fixed positions	Word-to-word distances	Rotational embeddings
Scalability	Limited to training length	Adapts to long sequences	Works well for very long texts
Task Suitability	Sentence classification, NER	Reasoning, text generation	Large-scale LLMs, NLP tasks
Implementation Complexity	Simple	Moderate	More complex

Final Thoughts

- **Use APE** if absolute positioning is important, like in **sentence-level tasks** .
- **Use RPE** when **word relationships matter more than absolute positions** , like in **translation or reasoning tasks** .
- **Use RoPE** for **scalable, general-purpose NLP models** , especially in **large transformers** .

🚀 **Modern models like GPT-4 and LLaMA use RoPE** because it provides **flexibility, efficiency, and generalization** in large-scale language models.

Key Differences & Summary

Feature	Absolute (APE - BERT, ALBERT)	Relative (RPE - T5, Transformer-XL)	Rotary (RoPE - GPT-4, Llama)
Positional Reference	Fixed positions	Word-to-word distances	Rotational embeddings
Scalability	Limited to training length	Adapts to long sequences	Works well for very long texts
Task Suitability	Sentence classification, NER	Reasoning, text generation	Large-scale LLMs, NLP tasks
Implementation Complexity	Simple	Moderate	More complex

Introducing Self-Attention: The Intuition

1

Query

What a token is looking for

2

Key

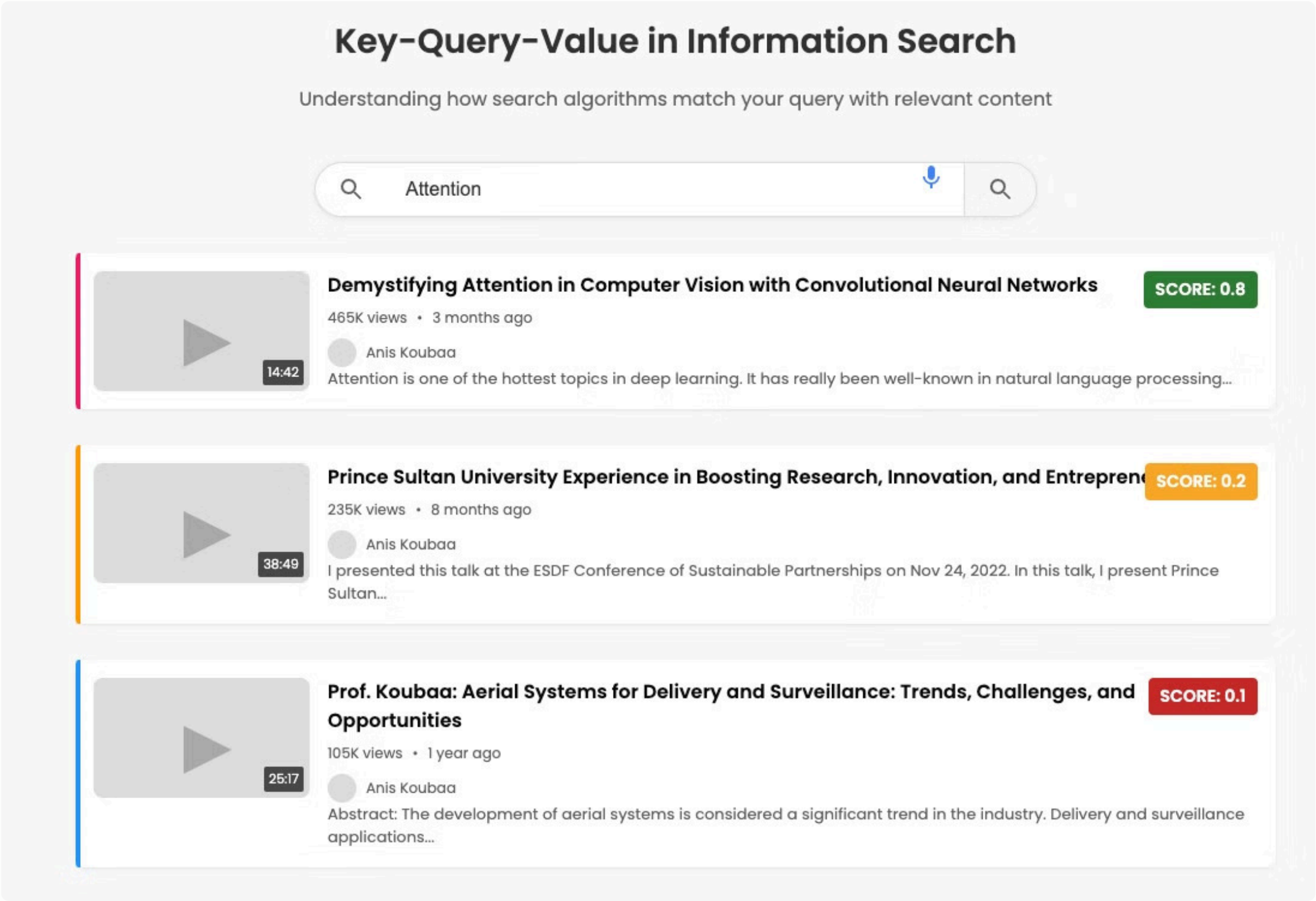
What a token offers

3

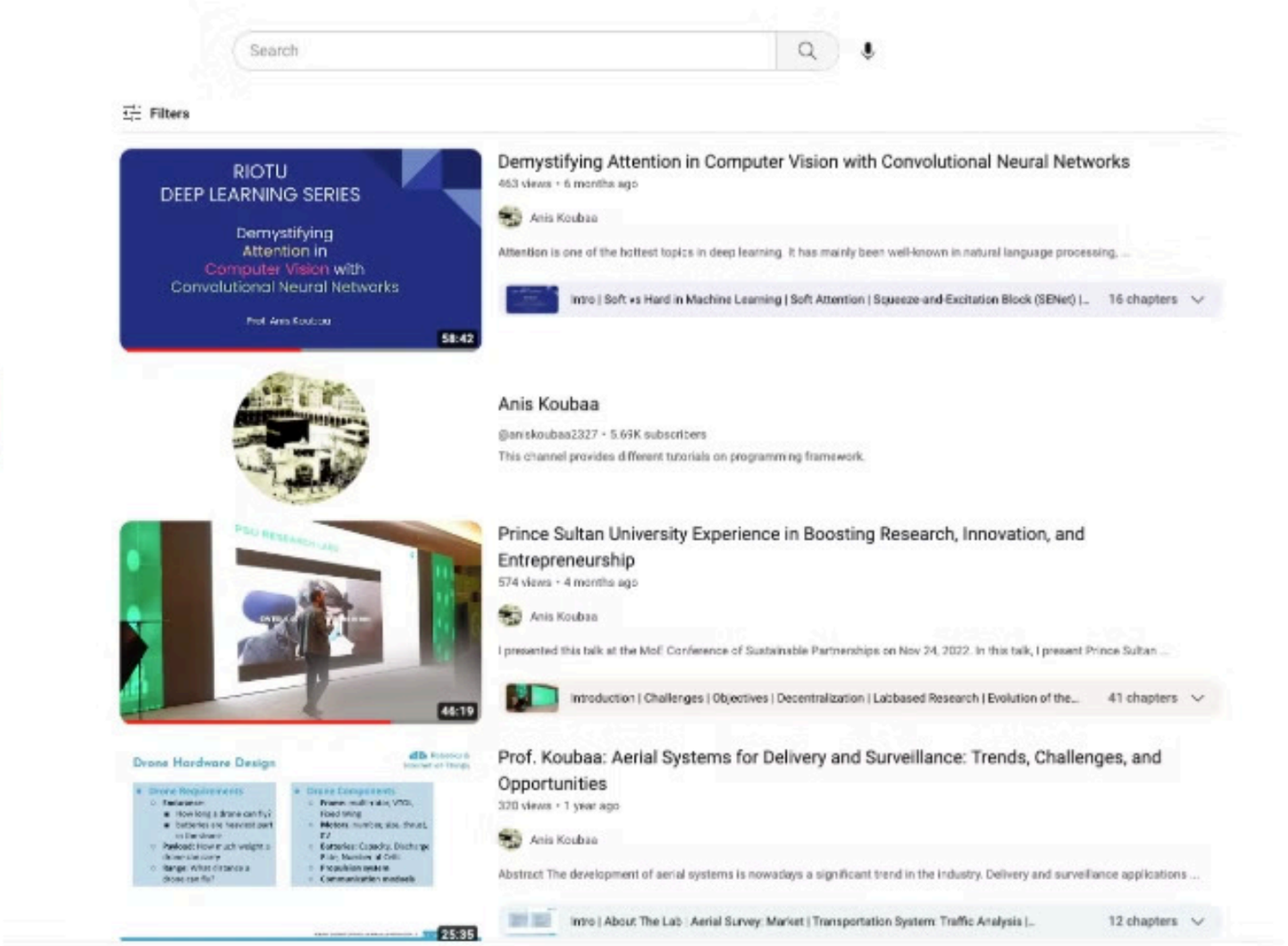
Value

The content of a token

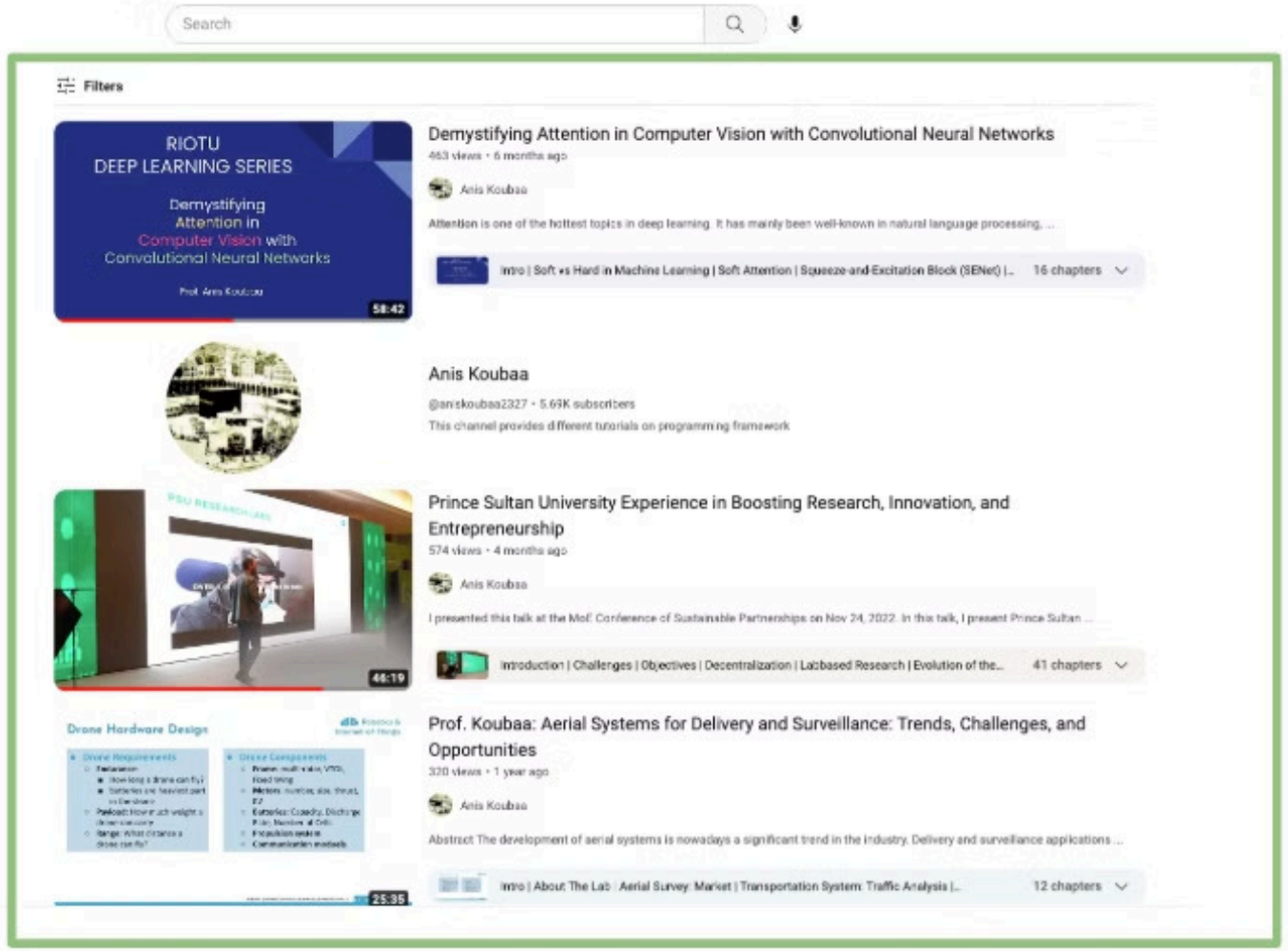
Intuition Behind Self-Attention



Intuition Behind Self-Attention

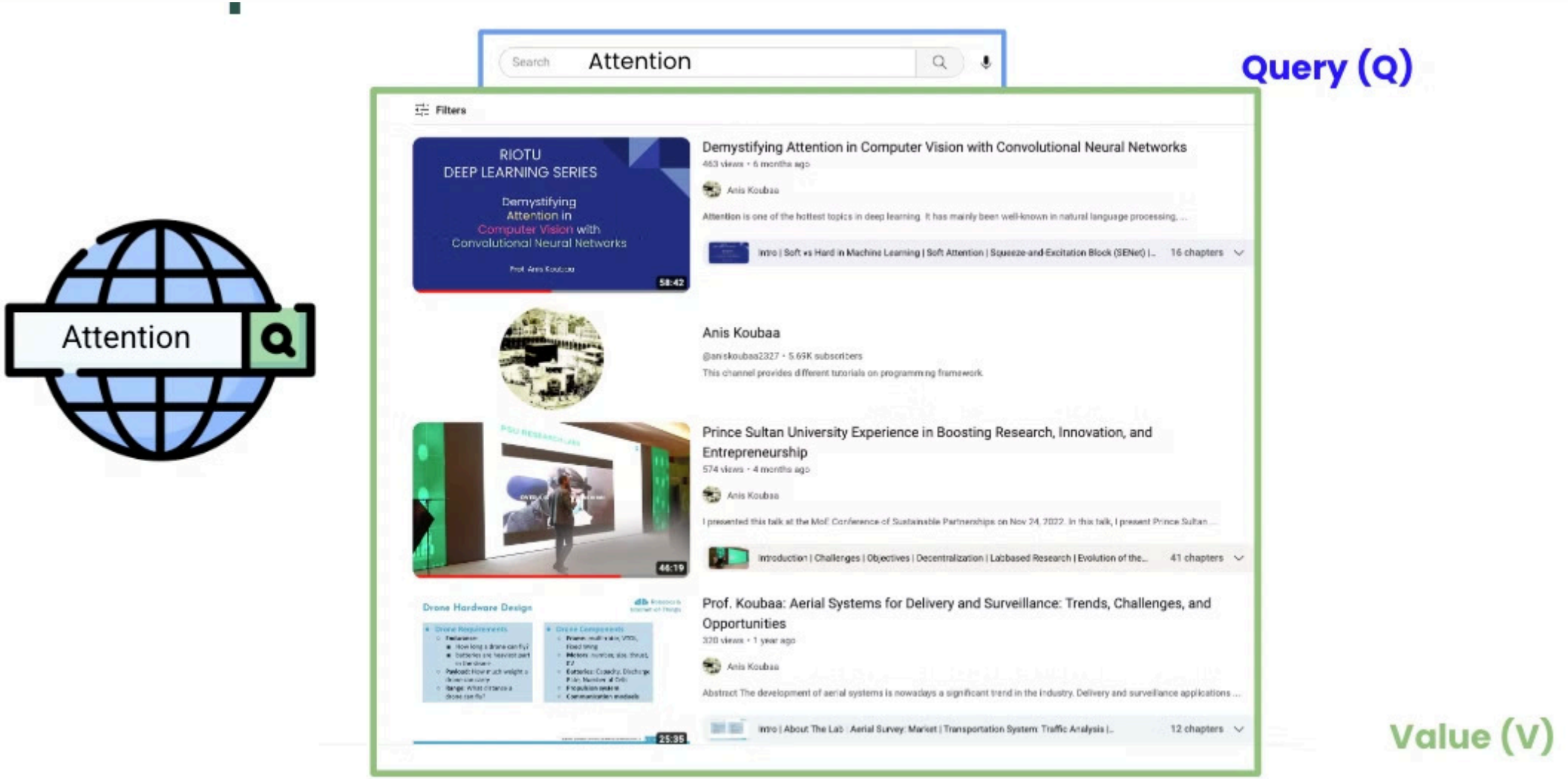


Intuition Behind Self-Attention

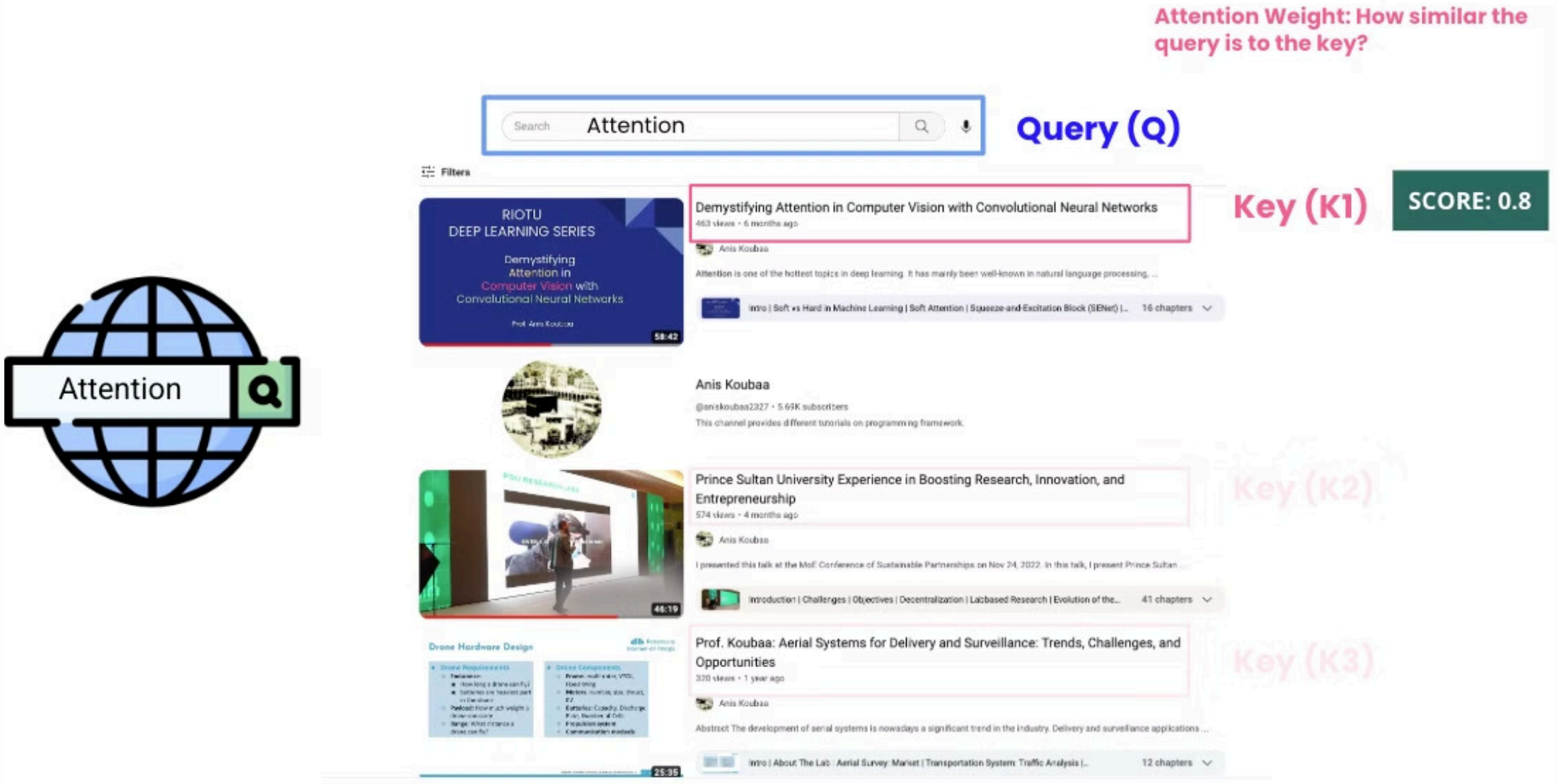


Value (V)

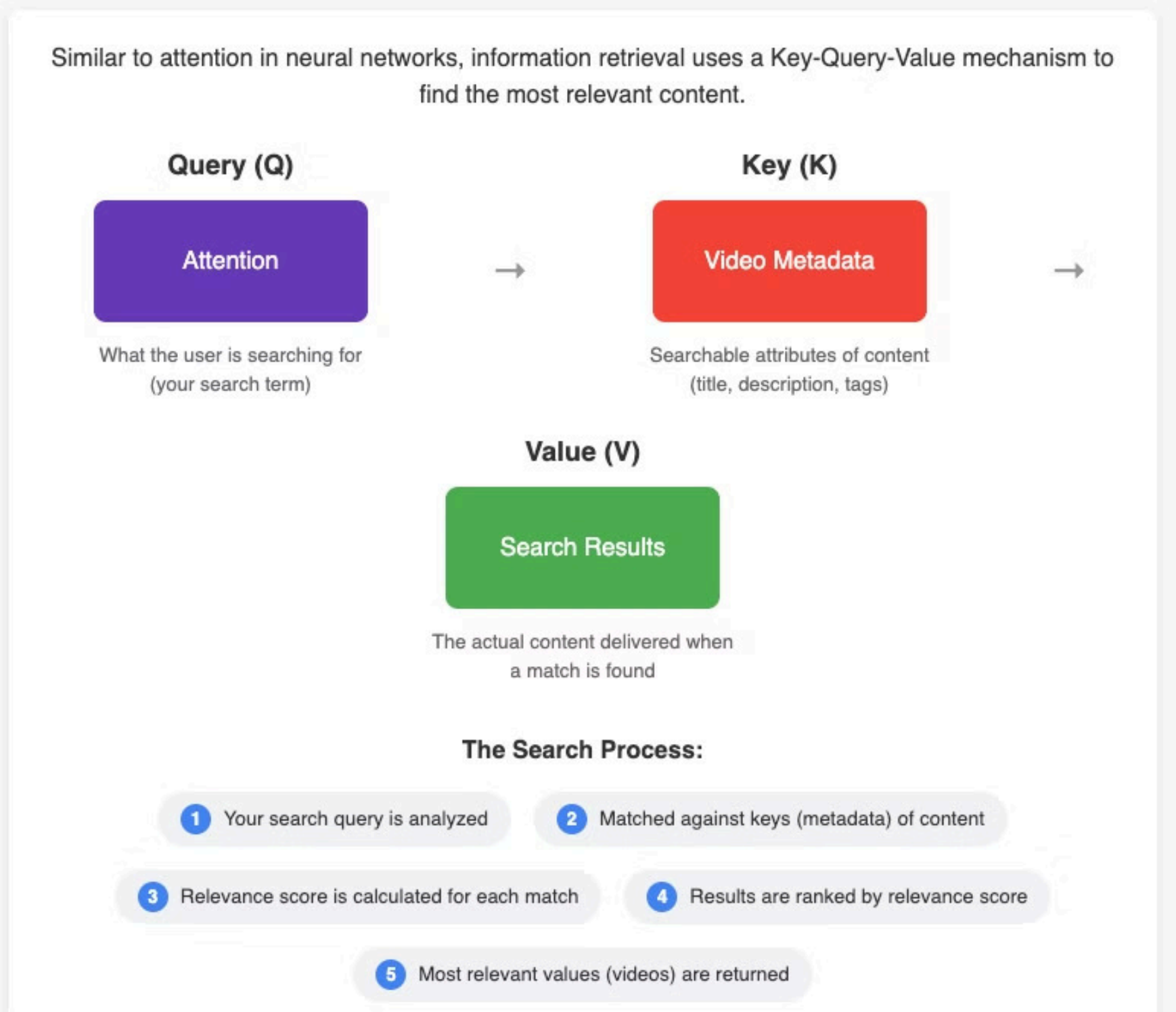
Intuition Behind Self-Attention



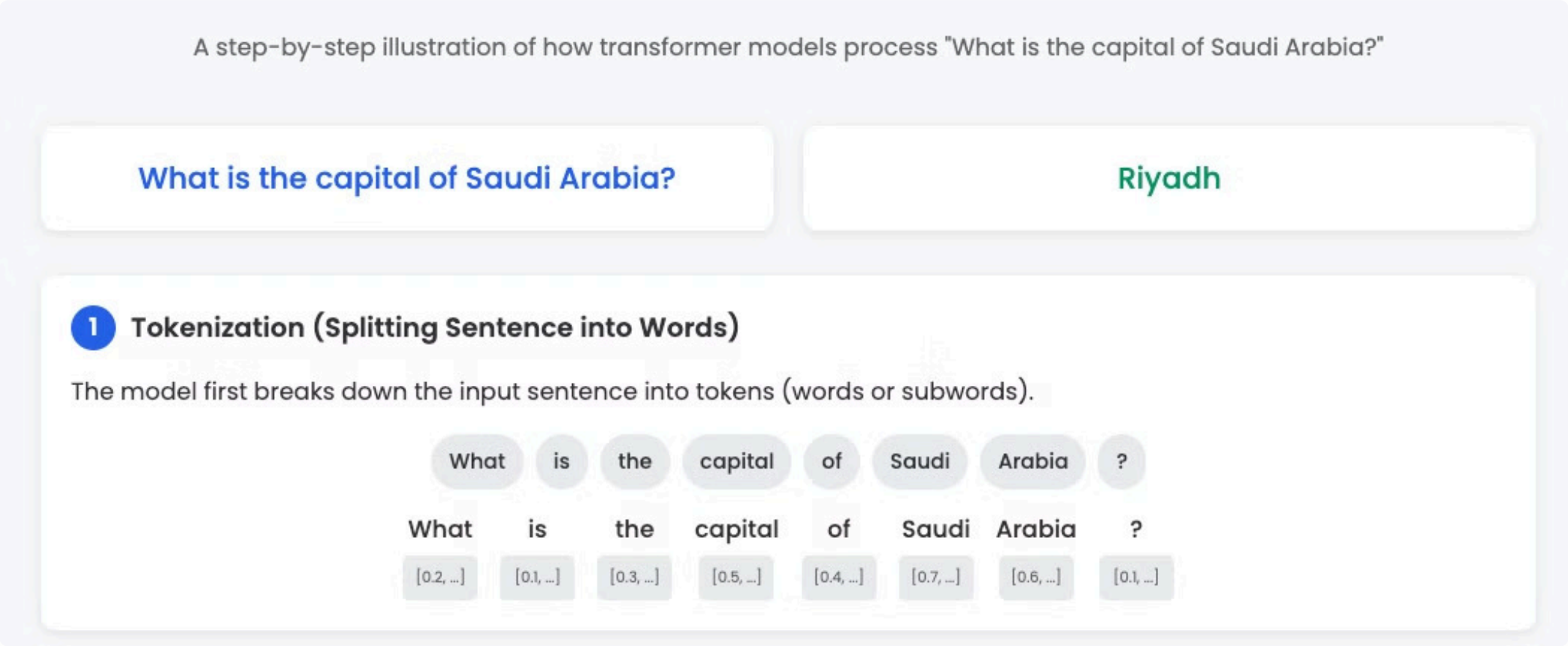
Intuition Behind Self-Attention



How Key-Query-Value (KQV) Mechanism Works in Search



How Self-Attention Makes Inference



How Self-Attention Makes Inference

1 Tokenization (Splitting Sentence into Words)

The model first breaks down the input sentence into tokens (words or subwords).

2 Generating Key (K), Query (Q), and Value (V)

Each token embedding is transformed into three different vectors: Query (Q), Key (K), and Value (V).

For the token "capital":

For the token "Saudi Arabia":

How Self-Attention Makes Inference

2 Generating Key (K), Query (Q), and Value (V)

Each token is transformed into three different vectors for self-attention:

For the token "capital":

Query (Q):

Key (K):

Value (V):

What these vectors represent:

- **Query (Q):** What information the token "capital" is looking for
- **Key (K):** What information the token "capital" can match with other tokens
- **Value (V):** The actual information content of "capital" to be used in the output

How Self-Attention Makes Inference

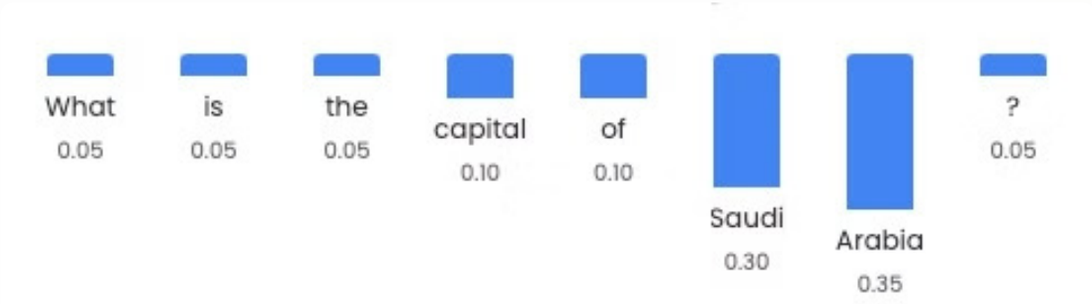
3 Computing Attention Scores

The model calculates how much each token should attend to others by computing dot products between queries and keys.

Attention Score Matrix (Q·K^T)

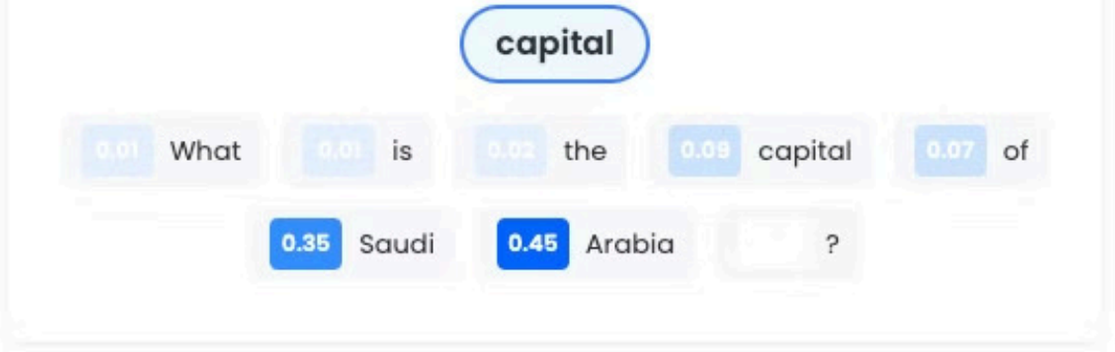
	What	is	the	capital	of	Saudi	Arabia	?
What	0.9	0.4	0.2	0.5	0.2	0.2	0.2	0.3
is	0.3	0.9	0.3	0.3	0.3	0.1	0.1	0.1
the	0.1	0.3	0.9	0.5	0.2	0.1	0.1	0.1
capital	0.1	0.2	0.3	0.9	0.5	0.7	0.8	0.1
of	0.1	0.2	0.2	0.5	0.9	0.6	0.5	0.1
Saudi	0.1	0.1	0.1	0.4	0.5	1.0	0.9	0.1
Arabia	0.1	0.1	0.1	0.4	0.4	0.8	1.0	0.2
?	0.5	0.2	0.1	0.2	0.1	0.1	0.2	1.0

How Self-Attention Makes Inference



4 Normalizing Attention Weights

A softmax function is applied to convert scores into probabilities that sum to 1.



How Self-Attention Makes Inference

5 Computing the Contextualized Word Representation

The model combines the values (V) of words based on attention weights to create context-aware representations.

After Self-Attention:

What is the capital of Saudi Arabia ?

The representation of "capital" is now enriched with information from "Saudi Arabia", and vice versa. This allows the model to understand that we're asking about "the capital of Saudi Arabia".

How Self-Attention Makes Inference

6 Generating the Answer

Finally, the model uses these enriched representations to generate the answer.

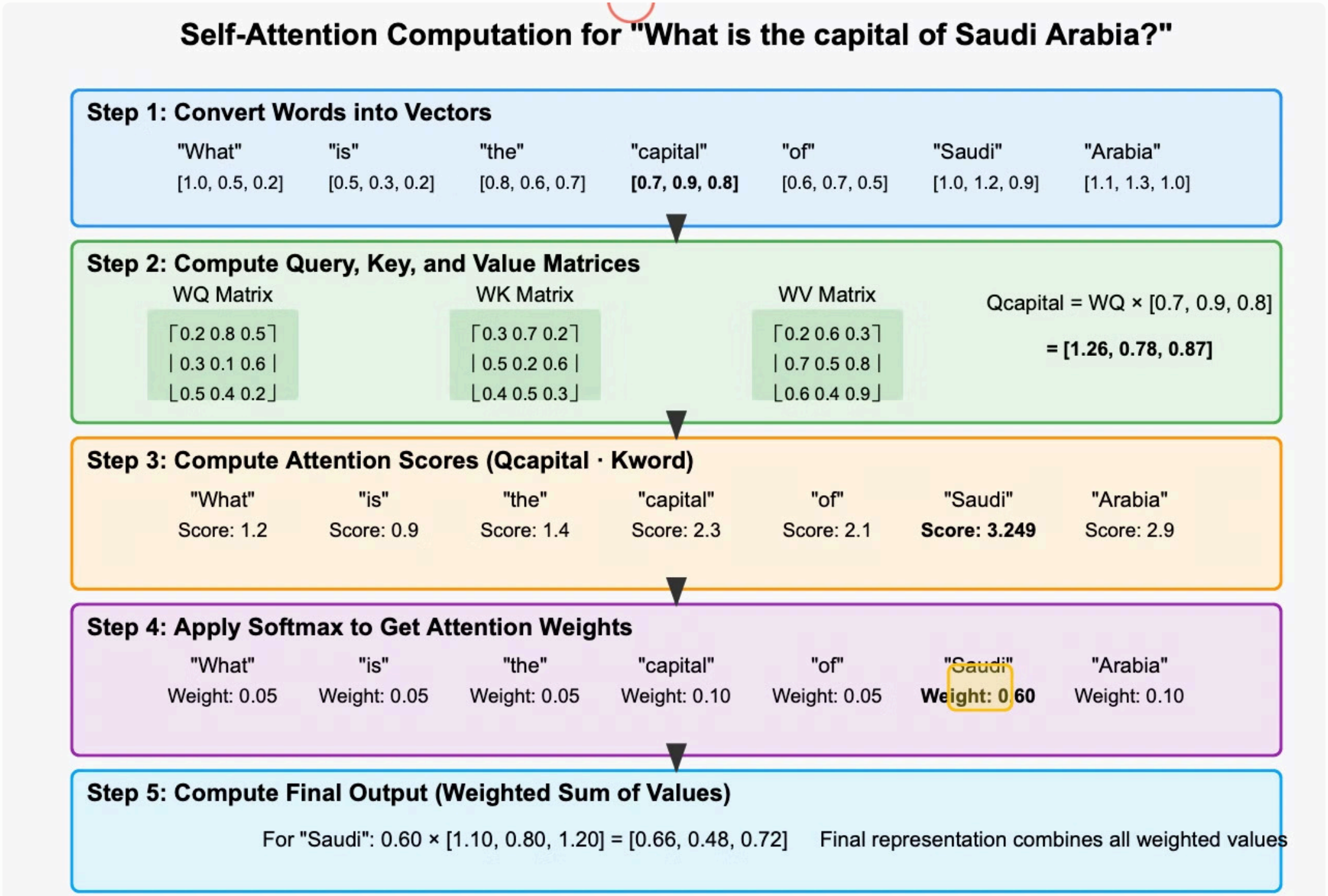
Based on the contextualized representations, the model now knows:

- "capital" is strongly connected to "Saudi Arabia"
- The query is asking for the capital of Saudi Arabia
- From its training, it knows the answer is:

Riyadh

Through self-attention, the model was able to connect the relevant parts of the query, understand the relationship between "capital" and "Saudi Arabia", and retrieve the correct answer "Riyadh" from its learned knowledge.

Step by Step Calculations



Step by Step Calculations

1 Convert Words into Vectors

Each word in the input sentence is first converted into an embedding vector. For simplicity, we're using 3-dimensional embeddings.

Word Embeddings (3-dimensional)

Word	Dimension 1	Dimension 2	Dimension 3	Vector Representation
What	1.0	0.5	0.2	[1.0, 0.5, 0.2]
is	0.5	0.3	0.2	[0.5, 0.3, 0.2]
the	0.8	0.6	0.7	[0.8, 0.6, 0.7]
capital	0.7	0.9	0.8	[0.7, 0.9, 0.8]
of	0.6	0.7	0.5	[0.6, 0.7, 0.5]
Saudi	1.0	1.2	0.9	[1.0, 1.2, 0.9]
Arabia	1.1	1.3	1.0	[1.1, 1.3, 1.0]

Step by Step Calculations

2 Define Query (Q), Key (K), and Value (V) Matrices

We need transformation matrices to convert word embeddings into Query (Q), Key (K), and Value (V) vectors. For simplicity, we'll use predefined transformation matrices:

Query Matrix (W^Q)

0.2	0.8	0.5
0.3	0.1	0.6
0.5	0.4	0.2

Key Matrix (W^K)

0.3	0.7	0.2
0.5	0.2	0.6
0.4	0.5	0.3

Value Matrix (W^V)

0.2	0.6	0.3
0.7	0.5	0.8
0.6	0.4	0.9

The model transforms each word embedding into three different vectors:

```
Query (Q): Word embedding × W_Q (weight matrix)
  Key (K): Word embedding × W_K (weight matrix)
Value (V): Word embedding × W_V (weight matrix)
```

For our example, let's assume the following transformation matrices (in practice, these would be learned during training):

Word	Query Vector (Q)	Key Vector (K)	Value Vector (V)
What	[1.1, 0.6, 0.3]	[1.0, 0.5, 0.2]	[0.9, 0.4, 0.1]
is	[0.6, 0.4, 0.3]	[0.5, 0.3, 0.2]	[0.4, 0.2, 0.1]
the	[0.9, 0.7, 0.8]	[0.8, 0.6, 0.7]	[0.7, 0.5, 0.6]
capital	[0.8, 1.0, 0.9]	[0.7, 0.9, 0.8]	[0.6, 0.8, 0.7]
of	[0.7, 0.8, 0.6]	[0.6, 0.7, 0.5]	[0.5, 0.6, 0.4]
Saudi	[1.1, 1.3, 1.0]	[1.0, 1.2, 0.9]	[0.9, 1.1, 0.8]
Arabia	[1.2, 1.4, 1.1]	[1.1, 1.3, 1.0]	[1.0, 1.2, 0.9]
?	[0.3, 0.4, 0.2]	[0.2, 0.3, 0.1]	[0.1, 0.2, 0.0]

Step by Step Calculations

3 Computing Query (Q), Key (K), and Value (V) Vectors

We apply these transformation matrices to each word embedding to generate Q, K, and V vectors. Let's demonstrate for the word "capital" with embedding [0.7, 0.9, 0.8]:

Computing $Q_{capital}$:

$$
\begin{aligned}
Q_{capital} &= W^Q \times [0.7, 0.9, 0.8] \\
&= [(0.2 \times 0.7) + (0.8 \times 0.9) + (0.5 \times 0.8), \\
&(0.3 \times 0.7) + (0.1 \times 0.9) + (0.6 \times 0.8), \\
&(0.5 \times 0.7) + (0.4 \times 0.9) + (0.2 \times 0.8)] \\
&= [0.14 + 0.72 + 0.40, 0.21 + 0.09 + 0.48, 0.35 + 0.36 + 0.16] \\
&= [1.26, 0.78, 0.87]
\end{aligned}
$$

Similarly, we can calculate the K vector for "Saudi":

Computing K_{Saudi}:

$$
\begin{aligned}
K_{Saudi} &= W^K \times [1.0, 1.2, 0.9] \\
&= [(0.3 \times 1.0) + (0.7 \times 1.2) + (0.2 \times 0.9), \\
&(0.5 \times 1.0) + (0.2 \times 1.2) + (0.6 \times 0.9), \\
&(0.4 \times 1.0) + (0.5 \times 1.2) + (0.3 \times 0.9)] \\
&= [0.3 + 0.84 + 0.18, 0.5 + 0.24 + 0.54, 0.4 + 0.6 + 0.27] \\
&= [1.32, 1.28, 1.27]
\end{aligned}
$$

And compute the V vector for "Saudi":

Computing V_{Saudi}:

$$
\begin{aligned}
V_{Saudi} &= W^V \times [1.0, 1.2, 0.9] \\
&= [(0.2 \times 1.0) + (0.6 \times 1.2) + (0.3 \times 0.9), \\
&(0.7 \times 1.0) + (0.5 \times 1.2) + (0.8 \times 0.9), \\
&(0.6 \times 1.0) + (0.4 \times 1.2) + (0.9 \times 0.9)] \\
&= [0.2 + 0.72 + 0.27, 0.7 + 0.6 + 0.72, 0.6 + 0.48 + 0.81] \\
&= [1.19, 2.02, 1.89]
\end{aligned}
$$

Step-by-Step Computation of Q, K, V Vectors

For the word "capital" in "What is the capital of Saudi Arabia?"

1 Given Word Embedding for "capital"

We start with the embedding vector for the word "capital" from our embedding lookup table:

$$\begin{bmatrix} 0.7 \\ 0.9 \\ 0.8 \end{bmatrix}$$

Compute Query (Q) for "capital"

2 Compute Query (Q) for "capital"

The Query vector is computed by multiplying the word embedding with the Query transformation matrix (W^Q):

Query Matrix (W^Q):

$$\begin{bmatrix} 0.2 & 0.8 & 0.5 \\ 0.3 & 0.1 & 0.6 \\ 0.5 & 0.4 & 0.2 \end{bmatrix}$$

$$Q_{capital} = \begin{bmatrix} 0.2 & 0.8 & 0.5 \\ 0.3 & 0.1 & 0.6 \\ 0.5 & 0.4 & 0.2 \end{bmatrix} \times \begin{bmatrix} 0.7 \\ 0.9 \\ 0.8 \end{bmatrix}$$

```
= [(0.2 × 0.7) + (0.8 × 0.9) + (0.5 × 0.8),
(0.3 × 0.7) + (0.1 × 0.9) + (0.6 × 0.8),
(0.5 × 0.7) + (0.4 × 0.9) + (0.2 × 0.8)]

= [0.14 + 0.72 + 0.40,
0.21 + 0.09 + 0.48,
0.35 + 0.36 + 0.16]

= [1.26, 0.78, 0.87]
```

Thus, the Query (Q) vector for "capital" is:

$$Q_{capital} = [1.26, 0.78, 0.87]$$

Compute Key (K) for "capital"

3 Compute Key (K) for "capital"

The Key vector is computed by multiplying the word embedding with the Key transformation matrix (W^K):

Key Matrix (W^K):

$$\begin{bmatrix} 0.3 & 0.7 & 0.2 \\ 0.5 & 0.2 & 0.6 \\ 0.4 & 0.5 & 0.3 \end{bmatrix}$$

$$K_{capital} = \begin{bmatrix} 0.3 & 0.7 & 0.2 \\ 0.5 & 0.2 & 0.6 \\ 0.4 & 0.5 & 0.3 \end{bmatrix} \times \begin{bmatrix} 0.7 \\ 0.9 \\ 0.8 \end{bmatrix}$$

```
= [(0.3 × 0.7) + (0.7 × 0.9) + (0.2 × 0.8),
(0.5 × 0.7) + (0.2 × 0.9) + (0.6 × 0.8),
(0.4 × 0.7) + (0.5 × 0.9) + (0.3 × 0.8)]

= [0.21 + 0.63 + 0.16,
0.35 + 0.18 + 0.48,
0.28 + 0.45 + 0.24]

= [1.00, 1.01, 0.97]
```

Thus, the Key (K) vector for "capital" is:

$$K_{capital} = [1.00, 1.01, 0.97]$$

Compute Value (V) for "capital"

4 Compute Value (V) for "capital"

The Value vector is computed by multiplying the word embedding with the Value transformation matrix (W^V):

Value Matrix (W^V):

$$\begin{bmatrix} 0.2 & 0.6 & 0.3 \\ 0.7 & 0.5 & 0.8 \\ 0.6 & 0.4 & 0.9 \end{bmatrix}$$

$$V_{capital} = \begin{bmatrix} 0.2 & 0.6 & 0.3 \\ 0.7 & 0.5 & 0.8 \\ 0.6 & 0.4 & 0.9 \end{bmatrix} \times \begin{bmatrix} 0.7 \\ 0.9 \\ 0.8 \end{bmatrix}$$

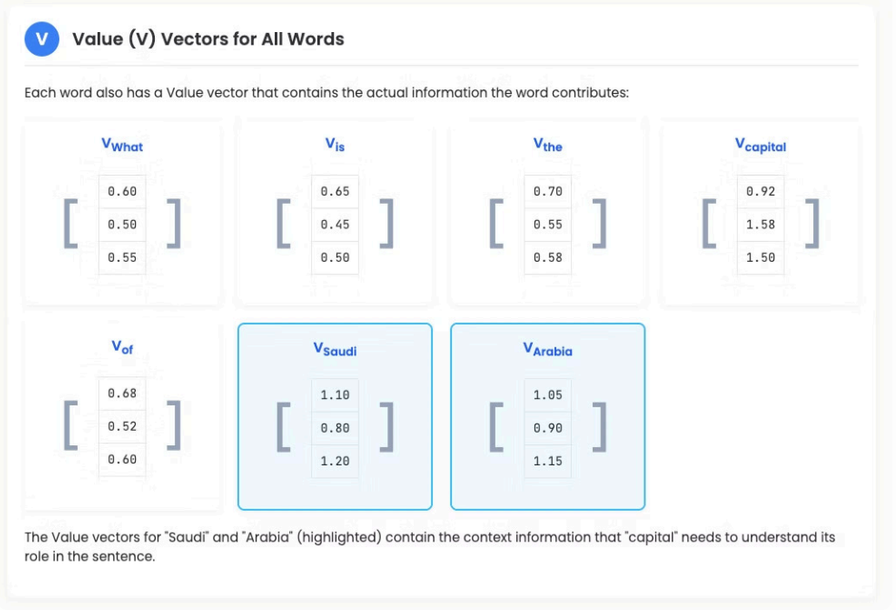
```
= [(0.2 × 0.7) + (0.6 × 0.9) + (0.3 × 0.8),
(0.7 × 0.7) + (0.5 × 0.9) + (0.8 × 0.8),
(0.6 × 0.7) + (0.4 × 0.9) + (0.9 × 0.8)]

= [0.14 + 0.54 + 0.24,
0.49 + 0.45 + 0.64,
0.42 + 0.36 + 0.72]

= [0.92, 1.58, 1.50]
```

Thus, the Value (V) vector for "capital" is:

$$V_{capital} = [0.92, 1.58, 1.50]$$

Query, Key, Value Matrices for Self-Attention

Focusing on "capital" as the Query Word in "What is the capital of Saudi Arabia?"



Matrix Representation of Self-Attention Key and Value

K Key (K) Matrix for All Words

The **Key (K) Matrix** organizes all Key vectors as columns, with each row representing one dimension of our embedding space.

$$K = \begin{bmatrix} \text{What} & \text{is} & \text{the} & \text{capital} & \text{of} & \text{Saudi} & \text{Arabia} \\ 0.92 & 0.75 & 0.80 & 1.00 & 0.72 & 1.20 & 1.10 \\ 0.65 & 0.55 & 0.68 & 1.01 & 0.60 & 1.00 & 1.15 \\ 0.70 & 0.60 & 0.72 & 0.97 & 0.65 & 1.10 & 1.05 \end{bmatrix}$$

Each column is a Key vector for a word. Each row represents one of the three dimensions. Note how "capital", "Saudi", and "Arabia" have similarly high values.

V Value (V) Matrix for All Words

The **Value (V) Matrix** organizes all Value vectors as columns, containing the actual contextual information each word contributes.

$$V = \begin{bmatrix} \text{What} & \text{is} & \text{the} & \text{capital} & \text{of} & \text{Saudi} & \text{Arabia} \\ 0.60 & 0.65 & 0.70 & 0.92 & 0.68 & 1.10 & 1.05 \\ 0.50 & 0.45 & 0.55 & 1.58 & 0.52 & 0.80 & 0.90 \\ 0.55 & 0.50 & 0.58 & 1.50 & 0.60 & 1.20 & 1.15 \end{bmatrix}$$

Each column is a Value vector for a word. These vectors contain the actual information each word contributes to the context.

Understanding Dot Product and Vector Similarity

5 Understanding Dot Product and Vector Similarity

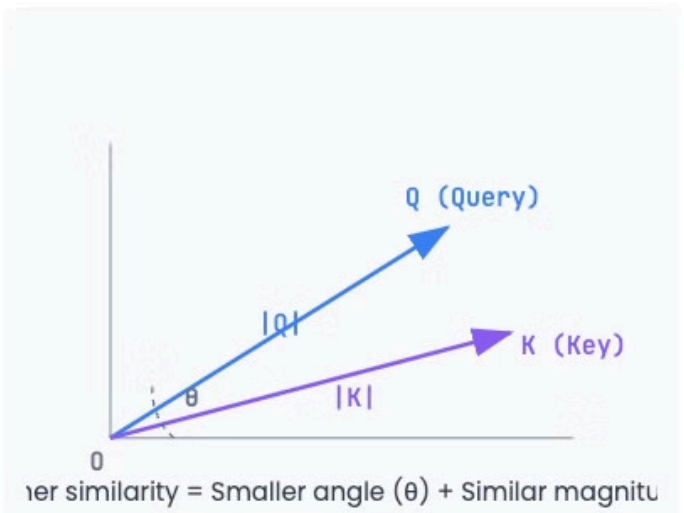
In self-attention, the **relevance scores** between words are calculated using the **dot product** between Query and Key vectors.

Dot Product: Measuring Vector Similarity

```
Dot Product of Vectors A and B = |A| × |B| × cos(θ)
```

Where:

- |A| and |B| are the magnitudes (lengths) of vectors A and B
- θ is the angle between the vectors
- cos(θ) ranges from -1 (opposite directions) to 1 (same direction)



ıer similarity = Smaller angle (θ) + Similar magnitu

$$Q \cdot K = |Q| \times |K| \times \cos(\theta)$$

Higher dot product → Higher similarity → Higher attention score

The dot product captures two important aspects of similarity:

Magnitude Similarity
Larger magnitudes in both vectors result in larger dot products. This rewards vectors that are both strong in similar dimensions.

Direction Similarity
The cosine term (cos θ) is largest when vectors point in the same direction, making the dot product a measure of directional alignment.

Understanding Dot Product and Vector Similarity

$$Q \cdot K = |Q| \times |K| \times \cos(\theta)$$

Higher dot product → Higher similarity → Higher attention score

The dot product captures two important aspects of similarity:

Magnitude Similarity

Larger magnitudes in both vectors result in larger dot products. This rewards vectors that are both strong in similar dimensions.

Direction Similarity

The cosine term (cos θ) is largest when vectors point in the same direction, making the dot product a measure of directional alignment.

Example: Relevance Score Calculation (capital → Saudi)

$$Q_{capital} \cdot K_{Saudi} = [1.26, 0.78, 0.87] \cdot [1.32, 1.28, 1.27]$$

$$= (1.26 \times 1.32) + (0.78 \times 1.28) + (0.87 \times 1.27)$$

$$= 1.66 + 1.00 + 1.11 = 3.77$$

Why Dot Product in Self-Attention?

- Words with similar semantic roles or contexts will have similar Query and Key vectors
- The dot product captures this similarity in both magnitude and direction
- Higher dot products (relevance scores) lead to higher attention weights
- This allows words to "pay attention" to other relevant words in the sequence

In our example:

The word "capital" has a high relevance score with "Saudi" (3.77) and "Arabia" (4.1) because:

1. The Query vector for "capital" and Key vectors for "Saudi" and "Arabia" have similar directions (small angle)
2. These vectors have relatively large magnitudes in complementary dimensions
3. This indicates semantic relevance - a capital is strongly related to its country

Computer Self-Attention Scores

4 Computing Attention Scores with Matrix Operations

To compute attention scores, we multiply the **Query vector** (transposed) with the **Key matrix**:

$$\text{Attention Scores} = Q_{capital}^{T} \times K$$

$$Q_{capital}^{T} = \begin{bmatrix} 1.26 & 0.78 & 0.87 \end{bmatrix} \times K = \begin{bmatrix} 0.92 & 0.75 & 0.80 & 1.00 & 0.72 & 1.20 & 1.10 \\ 0.65 & 0.55 & 0.68 & 1.01 & 0.60 & 1.00 & 1.15 \\ 0.70 & 0.60 & 0.72 & 0.97 & 0.65 & 1.10 & 1.05 \end{bmatrix}$$

↓

$$\text{Attention Scores} = \begin{bmatrix} \text{What} & \text{is} & \text{the} & \text{capital} & \text{of} & \text{Saudi} & \text{Arabia} \\ 2.10 & 1.73 & 1.91 & 2.62 & 1.65 & 3.25 & 3.27 \end{bmatrix}$$

```
For example, the attention score for "Saudi":
(1.26 × 1.20) + (0.78 × 1.00) + (0.87 × 1.10)
= 1.51 + 0.78 + 0.96
= 3.25

And for "Arabia":
(1.26 × 1.10) + (0.78 × 1.15) + (0.87 × 1.05)
= 1.39 + 0.90 + 0.91
= 3.27
```

The resulting row vector contains the attention scores between "capital" and each word in the sentence. Note how the scores for "Saudi" (3.25) and "Arabia" (3.27) are significantly higher.

Computer Self-Attention Scores

4 Compute Attention Scores

The attention score between two words is calculated as the dot product between the Query vector of one word and the Key vector of another word. We'll calculate attention scores between "capital" and all other words.

$$\text{Attention Score} = Q_{\text{capital}} \cdot K_{\text{word}}$$

$$\begin{aligned} Q_{\text{capital}} \cdot K_{\text{Saudi}} &= [1.26, 0.78, 0.87] \cdot [1.32, 1.28, 1.27] \\ &= (1.26 \times 1.32) + (0.78 \times 1.28) + (0.87 \times 1.27) \\ &= 1.66 + 1.00 + 1.11 = 3.77 \end{aligned}$$

Computing the same for all words, we get these attention scores:

What	is	the	capital	of
2.5	1.3	2.4	2.8	2.0

Saudi	Arabia
3.77	4.1

Note that the highest attention scores are for "Saudi" (3.77) and "Arabia" (4.1), which makes sense since "capital" should pay most attention to "Saudi Arabia" to understand the context.

Computer Scaled Self-Attention Scores

4 Compute Attention Scores

The attention score between two words is calculated as the dot product between the Query vector of one word and the Key vector of another word. We'll calculate attention scores between "capital" and all other words.

$$\text{Attention Score} = Q_{capital} \cdot K_{word} / \sqrt{d_k}$$

Where d_k is the dimension of the key vectors (in our case, $d_k = 3$). The scaling factor $\sqrt{d_k} = \sqrt{3} \approx 1.73$ prevents the dot products from growing too large in magnitude.

$$Q_{capital} \cdot K_{Saudi} / \sqrt{d_k} = [1.26, 0.78, 0.87] \cdot [1.32, 1.28, 1.27] / 1.73$$
$$= ((1.26 \times 1.32) + (0.78 \times 1.28) + (0.87 \times 1.27)) / 1.73$$
$$= (1.66 + 1.00 + 1.11) / 1.73 = 3.77 / 1.73 = 2.18$$

Computing the same for all words, we get these scaled attention scores:

What	is	the	capital	of
1.45	0.75	1.39	1.62	1.16

Saudi	Arabia
2.18	2.37

Note that the highest attention scores are for "Saudi" (2.18) and "Arabia" (2.37), which makes sense since "capital" should pay most attention to "Saudi Arabia" to understand the context.

Apply Softmax to Get Attention Weights

5 Apply Softmax to Get Attention Weights

To convert the raw attention scores into attention weights (probabilities that sum to 1), we apply the softmax function:

$$\text{softmax}(z_i) = e^{z_i} / \Sigma\, e^{z_j}$$

First, we calculate the exponential of each score:

Word	Scaled Score	e^{Score}
What	1.45	4.26
is	0.75	2.12
the	1.39	4.01
capital	1.62	5.05
of	1.16	3.19
Saudi	2.18	8.85
Arabia	2.37	10.70
Sum	-	38.18

Then, we divide each exponential by their sum to get the softmax probabilities:

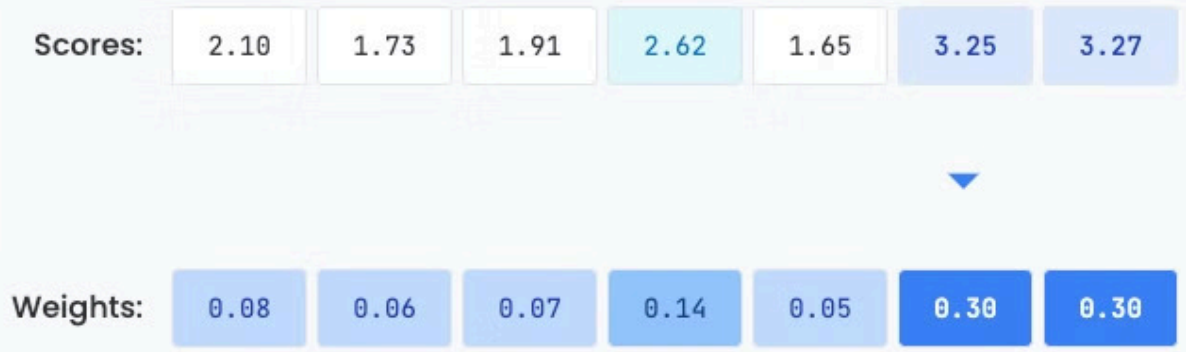
As expected, "Saudi" (0.23) and "Arabia" (0.27) receive the highest attention weights, together accounting for 50% of the attention from "capital".

Converting Scores to Weights with Softmax

5 Converting Scores to Weights with Softmax

We apply the **softmax function** to normalize the attention scores into a probability distribution:

```
Attention Weights = softmax(Attention Scores)
```



After applying softmax, we have a probability distribution that sums to 1. The words "Saudi" and "Arabia" receive the highest attention weights (30% each), indicating they provide the most relevant context for "capital".

Compute the Final Output

6 Compute the Final Output

Finally, we calculate the output by multiplying each word's Value (V) vector by its corresponding attention weight and summing them up.

$$\text{Output} = \Sigma\ (\text{attention weight} \times V_{word})$$

For "Saudi" with attention weight 0.23:

$$0.23 \times [1.19, 2.02, 1.89] = [0.27, 0.47, 0.44]$$

For "Arabia" with attention weight 0.27:

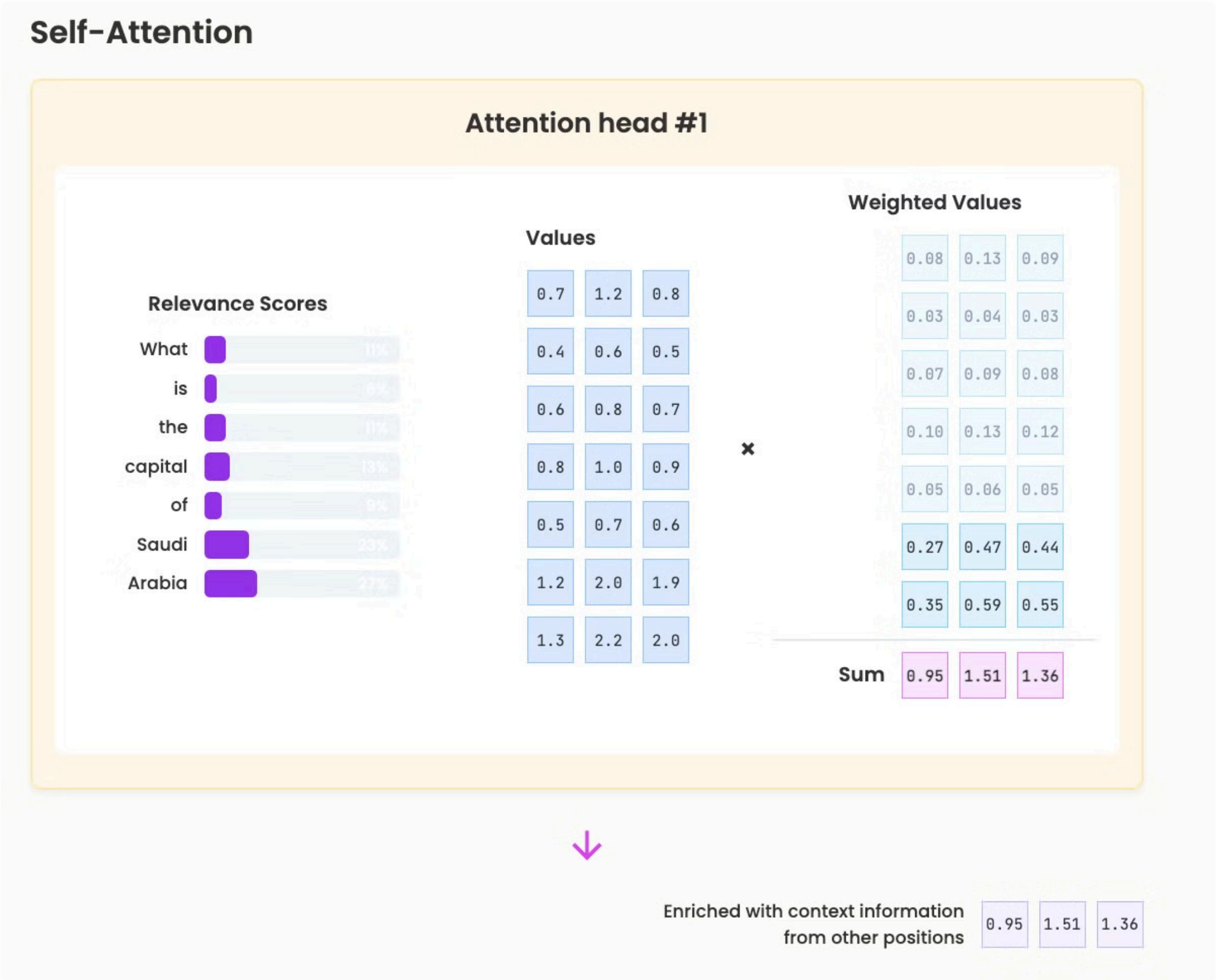
$$0.27 \times [1.30, 2.20, 2.03] = [0.35, 0.59, 0.55]$$

Similarly, we calculate the weighted contributions of all other words.

Final Output Vector (Context-Enriched Representation)

$$[0.62, 1.06, 0.99]$$

Compute the Final Output



Computing the Context-Enriched Representation

Computing the Context-Enriched Representation

Finally, we use the attention weights to compute a weighted sum of the Value vectors:

$$\text{Context Vector} = \text{Attention Weights} \times V^T$$

$$\begin{bmatrix} 0.08 & 0.06 & 0.07 & 0.14 & 0.05 & \mathbf{0.30} & \mathbf{0.30} \end{bmatrix} \times \begin{bmatrix} 0.60 & 0.65 & 0.70 & 0.92 & 0.68 & 1.10 & 1.05 \\ 0.50 & 0.45 & 0.55 & 1.58 & 0.52 & 0.80 & 0.90 \\ 0.55 & 0.50 & 0.58 & 1.50 & 0.60 & 1.20 & 1.15 \end{bmatrix} = \begin{bmatrix} 0.93 \\ 0.88 \\ 1.02 \end{bmatrix}$$

The result is a context-enriched representation for "capital" that heavily draws from the Value vectors of "Saudi" and "Arabia".

Why This Matters:

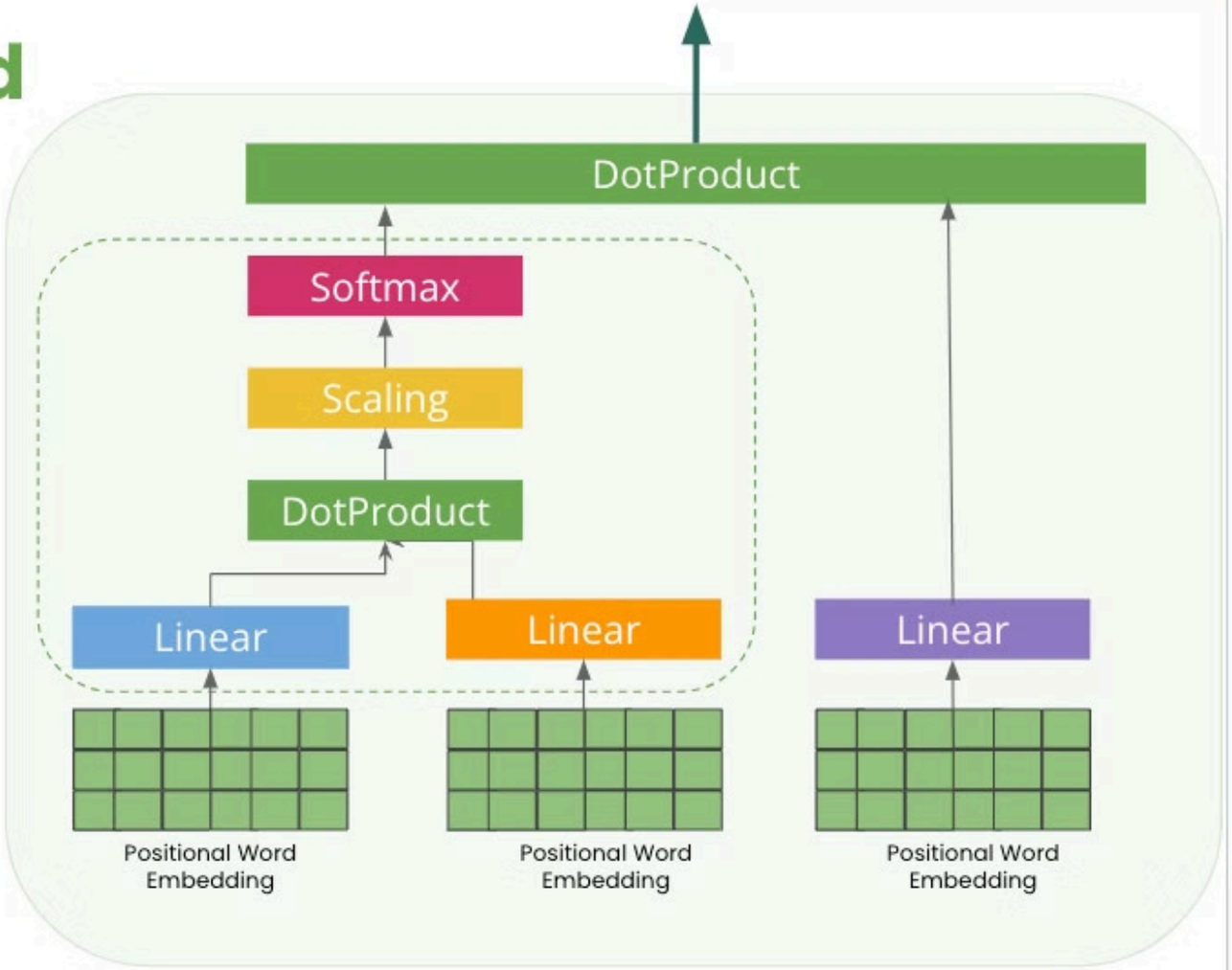
- **Efficient Matrix Operations:** Using matrices allows for parallel computation of attention across all words
- **Contextual Understanding:** The final vector for "capital" now includes information about "Saudi Arabia"
- **Learning Relationships:** The model learns which words should attend to others through training of the Q, K, V transformations
- **Connection to Language Understanding:** This mechanism enables the model to understand that the question is asking about Riyadh (the capital of Saudi Arabia)

Self-Attention Head

Self-Attention Head

```python
import torch
from torch import nn

def scaled_dot_product_attention(query: Tensor, key: Tensor, value: Tensor) -> Tensor:
    score = query.bmm(key.transpose(1, 2))
    scale = key.size(2) ** 0.5 #equivalent to scale = key.size(-1) ** 0.5
    scaled_score = score / scale
    softmax_score = funct.softmax(scaled_score, dim=-1)
    return softmax_score.bmm(value)

class AttentionHead(nn.Module):
    def __init__(self, dim_in: int, dim_q: int, dim_k: int):
        super().__init__()
        self.q = nn.Linear(dim_in, dim_q)
        self.k = nn.Linear(dim_in, dim_k)
        self.v = nn.Linear(dim_in, dim_k)

    def forward(self, query: Tensor, key: Tensor, value: Tensor) -> Tensor:
        return scaled_dot_product_attention(self.q(query), self.k(key), self.v(value))
```

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



Self-Attention Computation

Attention Formula

Attention(Q, K, V) = softmax(QK^T / sqrt(d_k))V

Complexity

Quadratic complexity: every token attends to every other token

Complexity Analysis of Self-Attention in Transformers

Self-Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$$

Where:

- **Q** = Query matrix (size N × d_k)
- **K** = Key matrix (size N × d_k)
- **V** = Value matrix (size N × d_v)
- **N** = Sequence length (number of tokens)
- **d_k** = Dimension of keys and queries
- **d_v** = Dimension of values (often equal to d_k)

Step-by-Step Self-Attention Computation

1 Compute QK^T

$Q_{N \times d_k} \times K^T_{d_k \times N} = S_{N \times N}$

Dot product between query and key vectors results in an N × N attention score matrix

2 Scale by $\sqrt{d_k}$

$S_{N \times N} \div \sqrt{d_k} = S'_{N \times N}$

Scaling prevents large values in longer sequences from creating extremely sharp softmax distributions

3 Apply Softmax

$\text{softmax}(S')_{N \times N} = A_{N \times N}$

Converts scores to probabilities, normalizing each row to sum to 1

4 Multiply with V

$A_{N \times N} \times V_{N \times d_v} = \text{Output}^{N \times d_v}$

Creates weighted representation of values for each token

Visual Illustration of N×N Attention Matrix

0.7	0.1	0.0	0.1	0.1
0.3	0.5	0.1	0.0	0.1
0.1	0.2	0.5	0.1	0.1
0.0	0.1	0.3	0.4	0.2
0.0	0.0	0.1	0.2	0.7

N×N attention weights matrix for N=5 tokens

Complexity Analysis

Complexity Analysis

Operation	Complexity	Explanation
Compute QK^T	$O(N^2 d_k)$	Matrix multiplication of $N \times d_k$ with $d_k \times N$ matrices
Scale by $\sqrt{d_k}$	$O(N^2)$	Element-wise division for $N \times N$ matrix
Apply Softmax	$O(N^2)$	Applied to each row of the $N \times N$ matrix
Multiply with V	$O(N^2 d_v)$	Matrix multiplication of $N \times N$ with $N \times d_v$ matrices

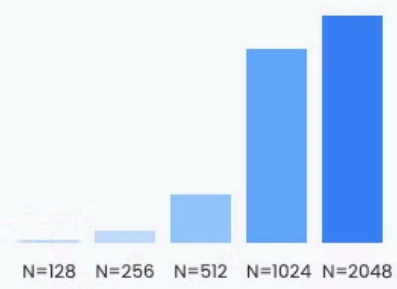
Memory Complexity:

- Storing the attention matrix requires $O(N^2)$ memory
- This becomes problematic for very long sequences

Scaling Issues:

Sequence Length (N)	Attention Matrix Size
128	16,384 elements
512	262,144 elements
1,024	1,048,576 elements
4,096	16,777,216 elements
16,384	268,435,456 elements

Growth Visualization:



Quadratic growth of computational requirements as sequence length increases

Why Does Self-Attention Have Quadratic Complexity?

Why Does Self-Attention Have Quadratic Complexity?

Pairwise Interactions:

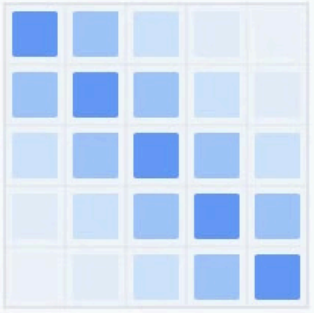
- Every token attends to every other token in the sequence
- Each token computes attention scores with all N tokens
- This creates an N×N attention matrix by design

Example:

If N=512 (common in BERT models), the attention matrix has:

- `512 × 512 = 262,144` elements per attention head
- With 12 attention heads and 12 layers, that's over `37 million` attention scores

All-to-All Attention Pattern



N×N attention matrix where every token attends to every other token

Why Is Quadratic Complexity a Problem?

Why Is Quadratic Complexity a Problem?

Memory Usage:

- The attention matrix grows as N^2 (quadratically)
- Storing these matrices consumes significant GPU/TPU memory
- Becomes impractical for very long sequences

Computational Cost:

- Large N leads to exponentially higher computation requirements
- Increased training and inference times
- Higher energy consumption and hardware costs

Real-world Example: GPT-3

- GPT-3 (175B parameters) uses sequence length N=2048
- Each attention matrix has: `2048 × 2048 = 4,194,304` elements
- That's over 4 million elements per layer per attention head
- With 96 attention heads and 96 layers in GPT-3, the total number of attention score computations is enormous

Increasing the context length to 8K or 16K would make the computation 16-64 times larger!

Why Is Quadratic Complexity a Problem?

Why Is Quadratic Complexity a Problem?

Memory Usage:

- The attention matrix grows as N^2 (quadratically)
- Storing these matrices consumes significant GPU/TPU memory
- Becomes impractical for very long sequences

Computational Cost:

- Large N leads to exponentially higher computation requirements
- Increased training and inference times
- Higher energy consumption and hardware costs

Real-world Example: GPT-3

- GPT-3 (175B parameters) uses sequence length N=2048
- Each attention matrix has: `2048 × 2048 = 4,194,304` elements
- That's over 4 million elements per layer per attention head
- With 96 attention heads and 96 layers in GPT-3, the total number of attention score computations is enormous

Increasing the context length to 8K or 16K would make the computation 16-64 times larger!

Total Time Complexity of Self-Attention:

$$O(N^2 d)$$

Since d (embedding dimension) is typically constant, the dominant term is:

$$O(N^2)$$

This quadratic complexity is why many recent research efforts focus on efficient attention mechanisms that reduce this to O(N log N) or even O(N).

Efficient Attention Mechanisms

Reducing the $O(N^2)$ complexity of standard self-attention

A. Sparse Attention

Key Idea

Instead of attending to **all** tokens in the sequence, **Sparse Attention** selects only a subset of relevant tokens, reducing unnecessary computations.

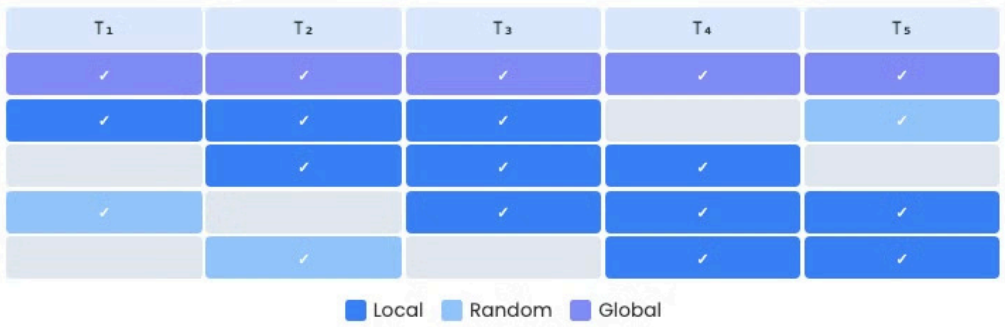
Longformer `O(N)`

- Uses **"dilated" sliding windows**
- Each token attends to a fixed-size neighborhood
- Good for long documents (legal texts, research papers)

BigBird `O(N)`

- Hybrid attention pattern:
- **Local** + **Random** + **Global** attention
- Ideal for very long sequences (books, DNA)

BigBird Attention Pattern



Standard Attention	vs	Sparse Attention
N^2 connections		$k \cdot N$ connections (k = window size)

B. Linformer (Linear Attention)

Key Idea

Instead of storing **N** keys and values, Linformer **compresses** them into a smaller space with **dimensionality reduction**.

How It Works

$$\text{Standard: Attention} = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Standard Attention Matrix: $O(N^2)$

A_{11}	A_{12}	A_{13}	A_{14}	A_{15}
A_{21}	A_{22}	A_{23}	A_{24}	A_{25}
A_{31}	A_{32}	A_{33}	A_{34}	A_{35}
A_{41}	A_{42}	A_{43}	A_{44}	A_{45}
A_{51}	A_{52}	A_{53}	A_{54}	A_{55}

Linformer: Project Keys & Values to lower dimension ($k \ll N$)

P_{11}	P_{12}	P_{13}	P_{14}	P_{15}
P_{21}	P_{22}	P_{23}	P_{24}	P_{25}

$$\text{Linformer: Attention} = \text{softmax}(Q(P_K K)^T / \sqrt{d_k}) \cdot (P_V V)$$

Benefits of Linformer

- Reduces complexity from $O(N^2)$ to $O(N \cdot d_k)$
- No significant loss of performance for most tasks
- Memory usage is greatly reduced for long sequences
- Works well for document processing requiring long context

$$O(N^2) \rightarrow O(N)$$

Makes longer contexts (10k+ tokens) feasible

Efficient Attention Mechanisms

Performer & Reformer:
Reducing attention complexity without sacrificing performance

C. Performer (Fast Attention via Kernels) $O(N \cdot d_k)$

Key Idea

Instead of computing **softmax(QK^T)** explicitly, **Performer approximates it using kernel functions**, allowing for linear-time computation.

How It Works

$$\text{Standard: Attention} = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Kernel-Based Approximation

$QK^T \rightarrow \text{softmax}(QK^T) \rightarrow N \times N \text{ matrix}$

Replace with kernel trick: $\varphi(Q)\varphi(K)^T$

$\varphi(Q) \times \varphi(K)^T V = \text{Linear complexity}$

FAVOR+ method uses orthogonal random features to approximate softmax attention

Standard Softmax	vs	Kernel Method
$O(N^2)$		$O(N \cdot d_k)$

Benefits of Performer

- **Linear complexity** in sequence length
- **Constant memory usage** regardless of sequence length
- Useful for **real-time applications** (robotics, RL)
- Backward-compatible with pretrained Transformer models
- Especially effective for **time-series forecasting**

D. Reformer (Locality-Sensitive Hashing) $O(N \cdot \log N)$

Key Idea

Reformer **replaces standard dot-product attention** with **Locality-Sensitive Hashing (LSH)**, making retrieval faster by only computing attention between similar items.

How It Works

Similar vectors are hashed to the same bucket - attention is only computed within buckets

Standard Attention	vs	LSH Attention
N^2 comparisons		$N \cdot \log N$ comparisons

Benefits of Reformer

- Reduces memory usage by up to **80%**
- Makes processing **very long sequences** feasible
- Ideal for **entire books or genome sequences**
- Efficiently handles **sparse attention patterns**
- Combined with reversible layers for additional memory savings

Complexity Comparison of Attention Mechanisms

Standard Attention	Reformer	Performer	Sparse Attention
$O(N^2)$	$O(N \cdot \log N)$	$O(N)$	$O(N)$
Full pairwise attention between all tokens	LSH-based approximation	Kernel-based approximation	Fixed pattern sparse attention

FlashAttention:

Optimizing GPU Memory for Faster Attention

What's the Big Idea?

The Problem with Standard Attention

Think of attention in Transformer models like a student trying to take notes during class:

- The student has a **small desk** (fast memory) where they can work quickly
- But they also have a **backpack** (slow memory) where they store most materials
- Standard attention keeps **going back to the backpack** for things they just put away
- This wastes time and makes learning slower

Work Smarter, Not Different

FlashAttention doesn't change what attention does—it just makes it more efficient

Fewer Trips to Memory

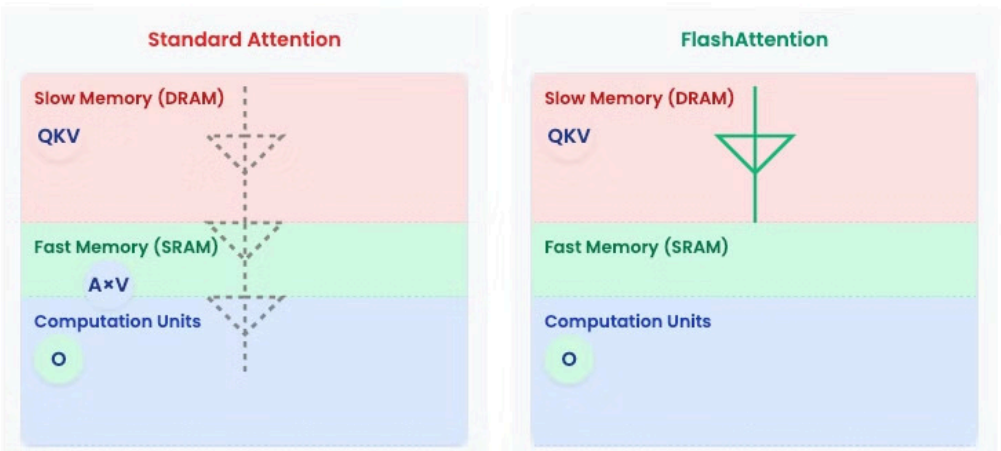
Process more data at once without constantly loading and saving

2-3× Faster

Same results but 2-3 times faster training and inference

How Does It Work?

Memory Flow Comparison



Many back-and-forth trips to slow memory

Minimal memory trips = faster processing

How it's like studying for an exam:

- **Standard attention** is like writing notes on small sticky notes and constantly sorting through them
- **FlashAttention** is like having a well-organized notebook where you can find information quickly
- Both approaches let you learn the same material, but the second one is much more efficient

Why Does This Matter?

	Standard Attention	FlashAttention
Training Speed	🦕	🚀
Memory Usage	📈 Grows with N²	📊 Grows with N
Max Sequence Length	📏 Limited	📏📏📏📏 4× Longer
Works with Existing Models	✅	✅

What This Means for AI

- **Faster innovation:** Researchers can train models 2-3× faster
- **Longer context:** Models can "remember" 4× more information
- **Lower costs:** Less GPU time = cheaper AI training
- **Same results:** No loss in accuracy or capability

Real-World Examples

🤖 GPT-4

🦙 LLaMA 2

🖼️ Stable Diffusion 3

FlashAttention:

A hardware-aware approach that speeds up attention computation without changing the mathematics

What is FlashAttention?

Key Idea

Instead of changing the attention mechanism itself (like sparse attention, Linformer, or Reformer), **FlashAttention optimizes how self-attention is computed in GPUs** by improving memory access patterns and reducing redundant operations.

Tiled Computation

Blocks of QKV matrices are processed in fast SRAM cache

Fused Operations

Combined softmax and matrix multiplication to reduce memory operations

Efficient I/O

Minimize data movement between GPU memory (DRAM) and compute units

Standard vs. FlashAttention

Memory Access Pattern Comparison

Standard Attention

- Load Q, K, V from DRAM
- Compute $S = QK^T$
- Store S to DRAM
- Load S from DRAM
- Compute softmax(S)
- Store P to DRAM
- Load P, V from DRAM
- Compute $O = PV$

FlashAttention

- Load blocks of Q, K, V
- Store in SRAM cache
- Compute S, softmax, and PV in one pass
- Accumulate partial results in SRAM
- Write final output O to DRAM

Performance Benefits & Applications

Aspect	Standard Attention	FlashAttention
Time Complexity	$O(N^2)$	$O(N^2)$
Memory Complexity	$O(N^2)$	$O(N)$
HBM Accesses	Many redundant reads/writes	Minimized I/O operations
Softmax Computation	Requires two passes over data	Single-pass online algorithm
Maximum Sequence Length	Limited by GPU memory	Much longer sequences possible

2-3× Faster Training

O(N) Memory Usage

4×+ Longer Sequences

FlashAttention:

A hardware-aware approach that speeds up attention computation without changing the mathematics

How FlashAttention Works

1. **Block-wise Processing:** Split Q, K, V matrices into smaller blocks that fit in fast SRAM memory
2. **Fused Operations:** Combine softmax normalization with matrix multiplication in a single kernel
3. **Recomputation:** Recompute attention scores when needed rather than storing the full N×N matrix
4. **Online Softmax:** Incrementally normalize attention scores to avoid materializing the full attention matrix

Where FlashAttention is Used

Large Language Models
LLaMA 2, GPT-4, PaLM

Text-to-Image Models
Stable Diffusion 3

Reinforcement Learning
Real-time RL, Robotics

Key Insight: FlashAttention doesn't change the attention mechanism's mathematics— it optimizes how it's implemented on GPUs, making it compatible with existing transformer models.

Standard vs. FlashAttention

Memory Access Pattern Comparison

Standard Attention	FlashAttention
Load Q, K, V from DRAM	Load blocks of Q, K, V
Compute $S = QK^T$	Store in SRAM cache
Store S to DRAM	Compute S, softmax, and PV in one pass
Load S from DRAM	Accumulate partial results in SRAM
Compute softmax(S)	Write final output O to DRAM
Store P to DRAM	
Load P, V from DRAM	
Compute O = PV	

Performance Benefits & Applications

Aspect	Standard Attention	FlashAttention
Time Complexity	$O(N^2)$	$O(N^2)$
Memory Complexity	$O(N^2)$	$O(N)$
HBM Accesses	Many redundant reads/writes	Minimized I/O operations
Softmax Computation	Requires two passes over data	Single-pass online algorithm
Maximum Sequence Length	Limited by GPU memory	Much longer sequences possible

2-3×
Faster Training

O(N)
Memory Usage

4×+
Longer Sequences

How FlashAttention Compares to Other Efficient Attention Methods

Comparison of Attention Optimization Methods

Method	Computational Complexity	Memory Usage	Optimization Strategy	Best Use Case
Standard Attention	$O(N^2)$	$O(N^2)$	Full pairwise token interaction	Small/medium sequences (≤ 512 tokens)
Sparse Attention (Longformer, BigBird)	$O(N)$	$O(N)$	Attends only to important tokens (local windows + global tokens)	Long documents, text classification
Linformer	$O(N \cdot d_k)$	$O(N)$	Compresses Key/Value matrices through low-rank approximation	NLP, text analysis, longer contexts
Performer	$O(N \cdot d_k)$	$O(N)$	Kernel-based approximation of softmax using random features	Fast real-time tasks, RL, robotics
Reformer	$O(N \cdot \log N)$	$O(N \cdot \log N)$	Local-Sensitive Hashing (LSH) to find similar queries/keys	Ultra-long texts, genomes
FlashAttention	$O(N^2)$	$O(N)$ (Key difference!)	Optimized memory access patterns & tiling algorithm that reduces HBM accesses	Training large-scale models (GPT-4, LLaMA)

Standard
Full N×N Attention

Sparse
Selective Attention

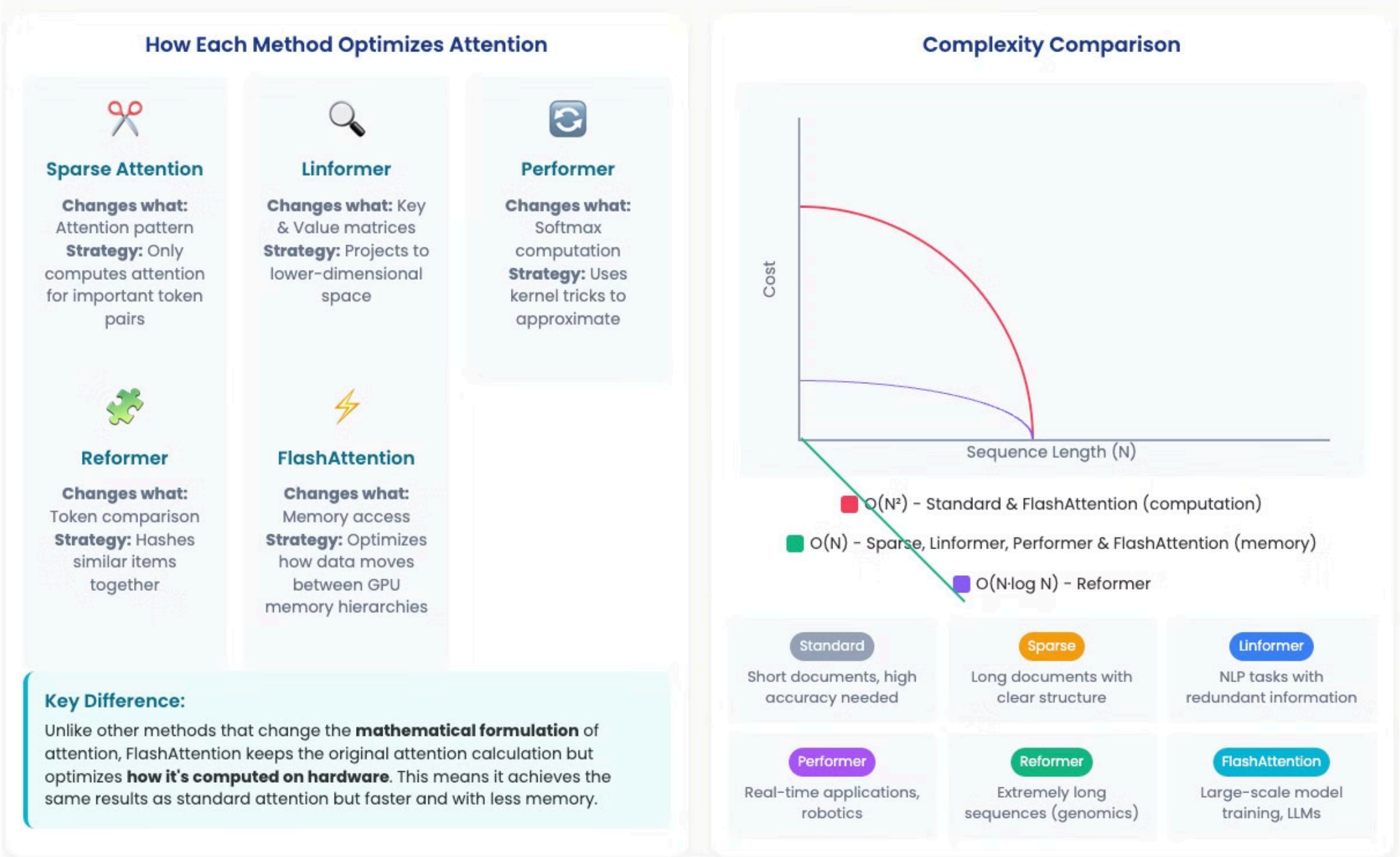
Linformer
Matrix Compression

Performer
Kernel Approximation

Reformer
Locality Hashing

FlashAttention
Memory Optimization

How FlashAttention Compares to Other Efficient Attention Methods



Masked Self-Attention

Causal Masks

Prevent "cheating" in autoregressive tasks (GPT)

Padding Masks

Used in encoder-only tasks

Masked Self-Attention in Transformers

Ensuring proper information flow with causal and padding masks

What is Masked Self-Attention?

Masked self-attention is a crucial modification of the standard self-attention mechanism in Transformer models, ensuring that information flows in a structured and appropriate way. There are two main types of masking:

Causal Masks (Future Masking)

Prevents tokens from attending to future tokens during generation, ensuring the model doesn't "cheat" by looking ahead.

Padding Masks

Prevents tokens from attending to padding tokens in batch processing, ensuring the model ignores irrelevant padding.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k} + M)V$$

M is the mask matrix, with different values for causal and padding masks

Masked Self-Attention in Transformers

Causal Masks (Future Masking)

Why Do We Need Causal Masks?

- In autoregressive models (like GPT), tokens are generated sequentially
- The model must not see future tokens when generating
- Without causal masks, the model would "cheat" by using information it shouldn't have access to yet

How Does Causal Masking Work?

The mask prevents each token from attending to future tokens by setting those attention scores to -∞. After softmax, these become 0, effectively blocking information flow from future tokens.

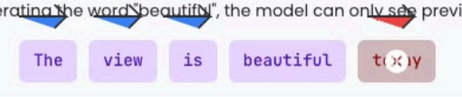
Causal Mask for a 5-token sequence

Token	t_1	t_2	t_3	t_4	t_5
t_1	0	-∞	-∞	-∞	-∞
t_2	0	0	-∞	-∞	-∞
t_3	0	0	0	-∞	-∞
t_4	0	0	0	0	-∞
t_5	0	0	0	0	0

$$M_{ij} = \{ 0, \text{if } j \leq i \text{ (allow past \& present tokens) } -\infty, \text{if } j > i \text{ (mask future tokens) } \}$$

Causal Mask in Action

When generating the word "beautiful", the model can only see previous words



Where Are Causal Masks Used?

GPT · LLaMA · Decoder-only Transformers · Machine Translation (Decoder)

Padding Masks

Why Do We Need Padding Masks?

- Sentences in natural language have different lengths
- Transformers process sentences in batches, requiring padding to maintain fixed dimensions
- Without padding masks, the model would attend to meaningless padding tokens

How Does Padding Masking Work?

Padding masks ensure that real tokens cannot attend to padding tokens by setting those attention scores to -∞. After softmax, these become 0, effectively preventing information flow from padding tokens.

Padding Mask for a batch with variable-length sequences

Token	t_1	t_2	t_3	PAD	PAD
t_1	0	0	0	-∞	-∞
t_2	0	0	0	-∞	-∞
t_3	0	0	0	-∞	-∞
PAD	-∞	-∞	-∞	-∞	-∞
PAD	-∞	-∞	-∞	-∞	-∞

$$M_{ij} = \{ 0, \text{if } j \text{ is a real token } -\infty, \text{if } j \text{ is a padding token } \}$$

Padding Mask in Action

Processing a shorter sentence with padding in a batch

Where Are Padding Masks Used?

BERT · RoBERTa · Encoder-only Transformers · Machine Translation (Encoder)

Masked Self-Attention in Transformers

Types of Attention Masks

Causal (Future) Mask
Prevents looking at future tokens

Padding Mask
Ignores padding tokens in batches

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k} + M)V$$

M is the mask matrix with different values for causal and padding masks

Causal Masks: Preventing "Cheating" in Generation

	t_1	t_2	t_3	t_4
t_1	0	−∞	−∞	−∞
t_2	0	0	−∞	−∞
t_3	0	0	0	−∞
t_4	0	0	0	0

Triangular mask: each position only sees itself and previous positions

Causal Mask in Action

The cat is sleeping

When predicting "is", model can see "The" and "cat" but not "sleeping"

- → Used in autoregressive text generation
- → Prevents "cheating" by looking ahead

$M_{ij} = \{ 0, \text{if } j \le i \text{ (past \& present)} \; -\infty, \text{if } j > i \text{ (future)} \}$

GPT LLaMA Decoder-only models

Padding Masks: Ignoring Irrelevant Padding

	t_1	t_2	t_3	PAD
t_1	0	0	0	−∞
t_2	0	0	0	−∞
t_3	0	0	0	−∞
PAD	−∞	−∞	−∞	−∞

Padding mask: real tokens attend to real tokens only, not padding

Padding Mask in Action

Hello world ! [PAD]

"Hello" can attend to real words but not to padding tokens

- → Used for variable-length sequences in batches
- → Prevents attending to meaningless padding

$M_{ij} = \{ 0, \text{if } j \text{ is a real token} \; -\infty, \text{if } j \text{ is a padding token} \}$

BERT RoBERTa Encoder-only models

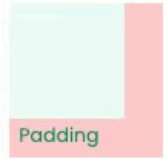
Comparison & Combined Usage

Comparison & Combined Usage

Mask Type	Purpose	Model Example	Prevents
Causal	Text generation	GPT, LLaMA	Looking at future tokens
Padding	Batch processing	BERT, RoBERTa	Attending to padding tokens

Visualization of Mask Types

Upper triangle masked



Padding columns/rows masked

Encoder (e.g., BERT)

real tokens here [PAD]

Uses **padding masks** only
Allows bidirectional attention between real tokens

Decoder (e.g., GPT)

can see future tokens

Uses **causal masks** only
Each token can only see previous tokens

Encoder-Decoder (e.g., T5, BART)

Encoder: padding mask + Decoder: causal mask

Combines both mask types: padding masks in encoder, causal masks in decoder

Masked Self-Attention in Python

selfattention.py

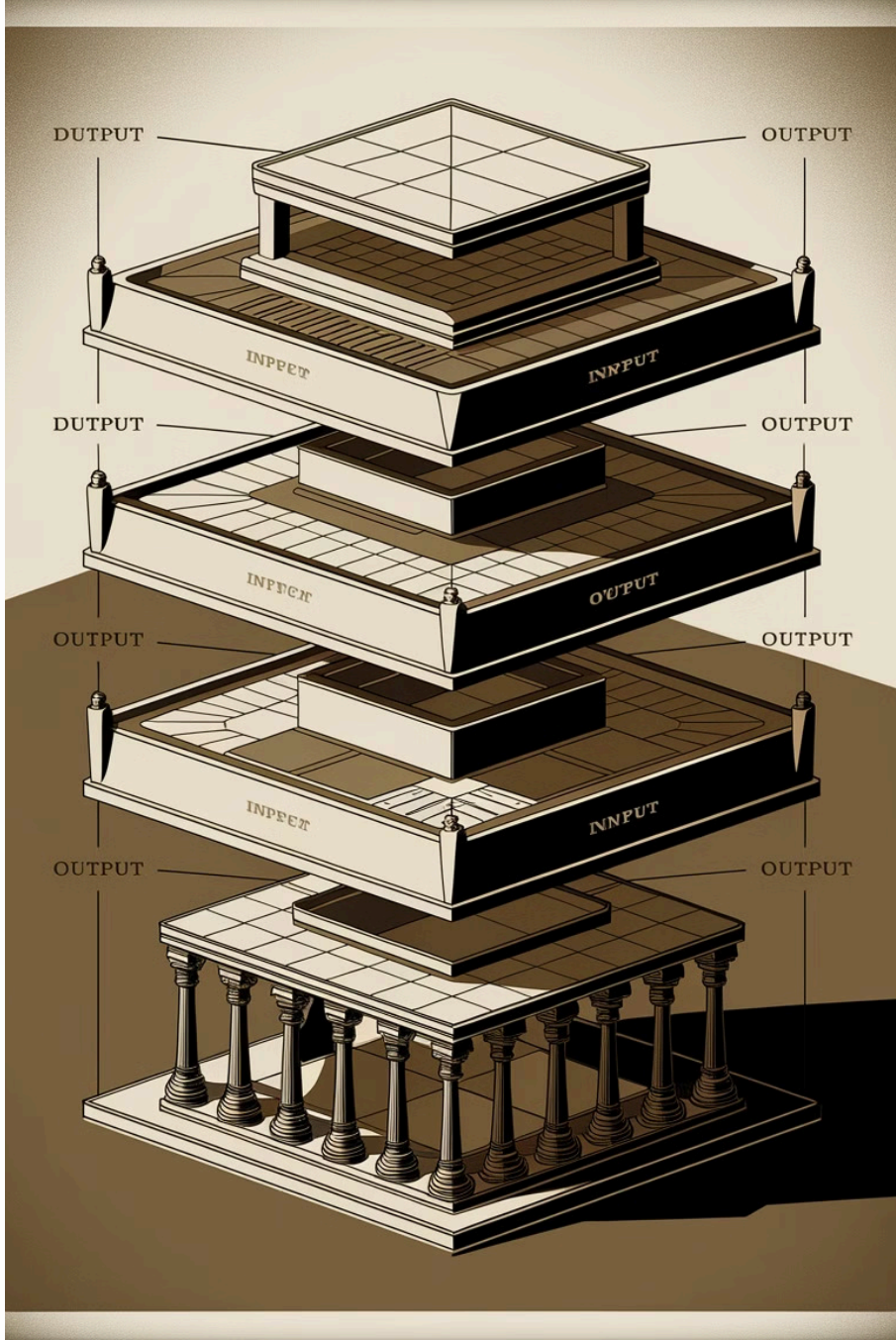
~/Desktop/selfattention.py

```python
from torch import nn

def scaled_masked_dot_product_attention(query: Tensor, key: Tensor, value: Tensor, mask: Tensor) -> Tensor:
    score = query.bmm(key.transpose(1, 2))
    scale = key.size(2) ** 0.5
    scaled_score = score / scale

    if mask is not None:
        scaled_score = scaled_score.masked_fill(mask == 0, -1e9)

    softmax_score = torch.nn.functional.softmax(scaled_score, dim=-1)
    return softmax_score.bmm(value)


class AttentionHead(nn.Module):
    def __init__(self, dim_in: int, dim_q: int, dim_k: int, mask: Tensor = None):
        super().__init__()
        self.q = nn.Linear(dim_in, dim_q)
        self.k = nn.Linear(dim_in, dim_k)
        self.v = nn.Linear(dim_in, dim_k)

    def forward(self, query: torch.Tensor, key: torch.Tensor, value: torch.Tensor, mask: torch.Tensor = None)
        return scaled_dot_product_attention(self.q(query), self.k(key), self.v(value), mask)

```

End of Lecture

Thank you for attending this lecture on Transformers.

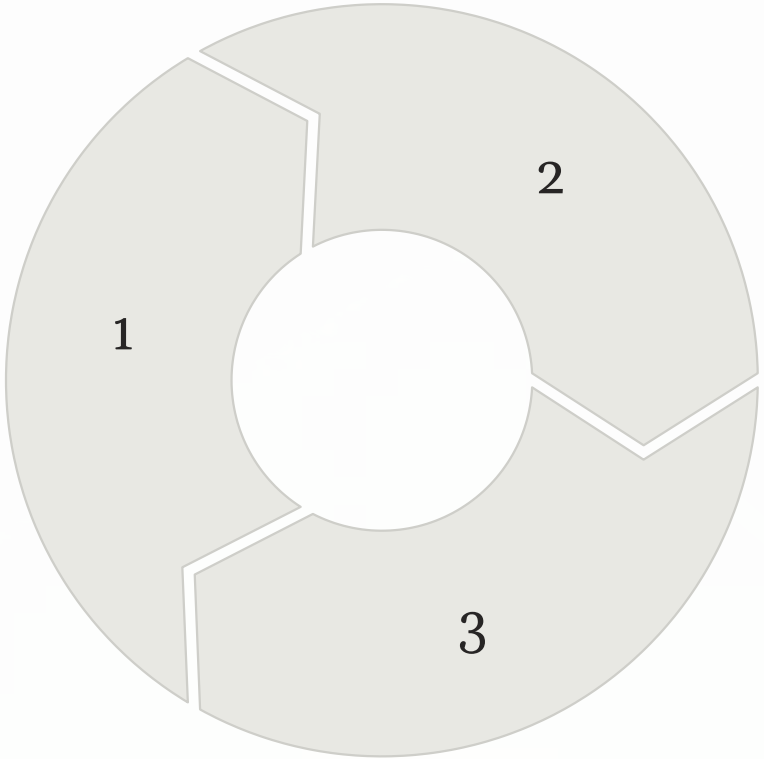
We hope you found the content informative.



Multi-Head Attention: Why Multiple Heads?

Diverse Representations

Each head processes a different subspace

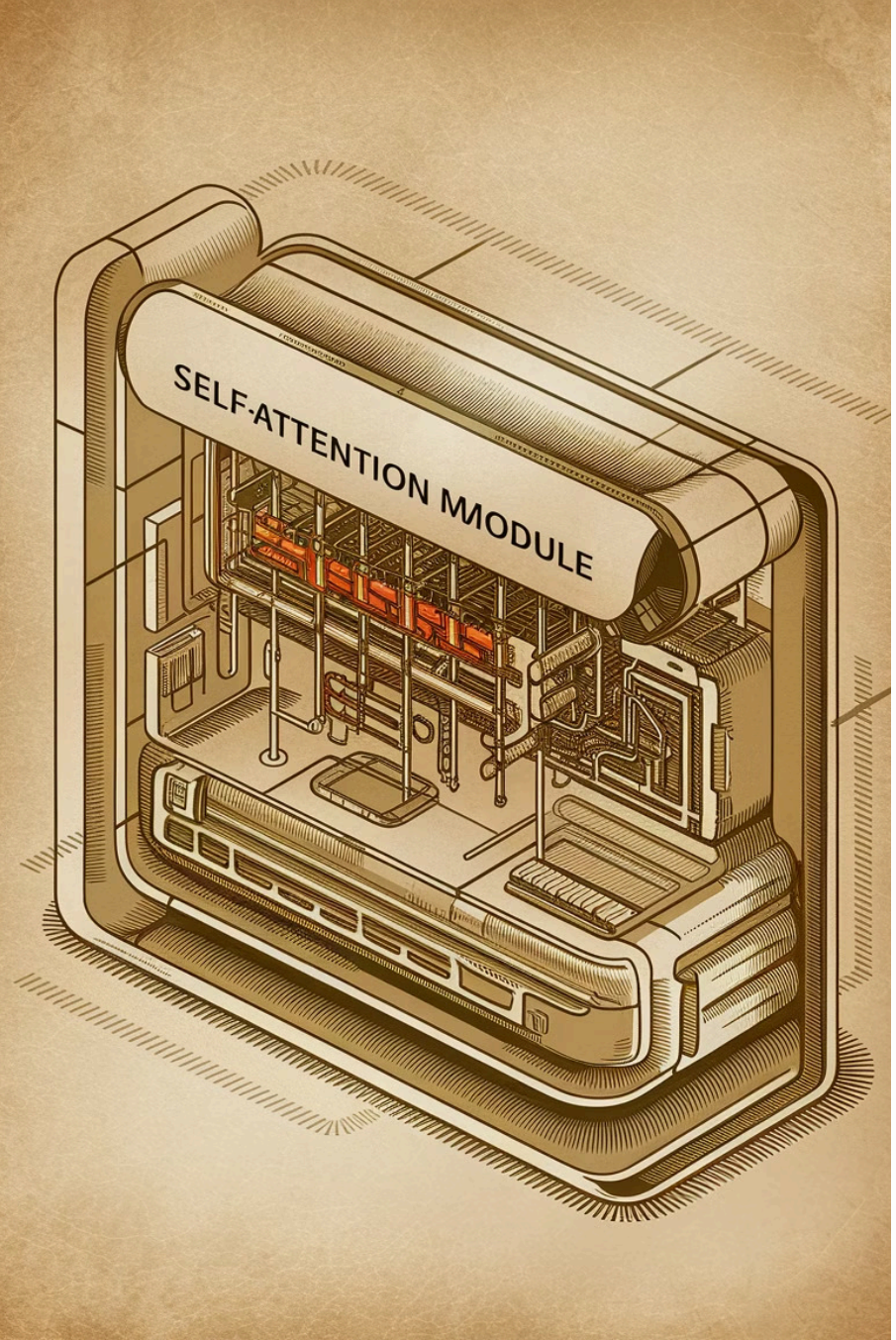


Capture Relationships

Syntactic vs. semantic aspects

Dimension Splitting

Embedding dimension divided among heads



Architecture of the Self-Attention Module

1

Linear Layers

For Q, K, V transformations

2

Concatenation

Combine multi-head outputs

3

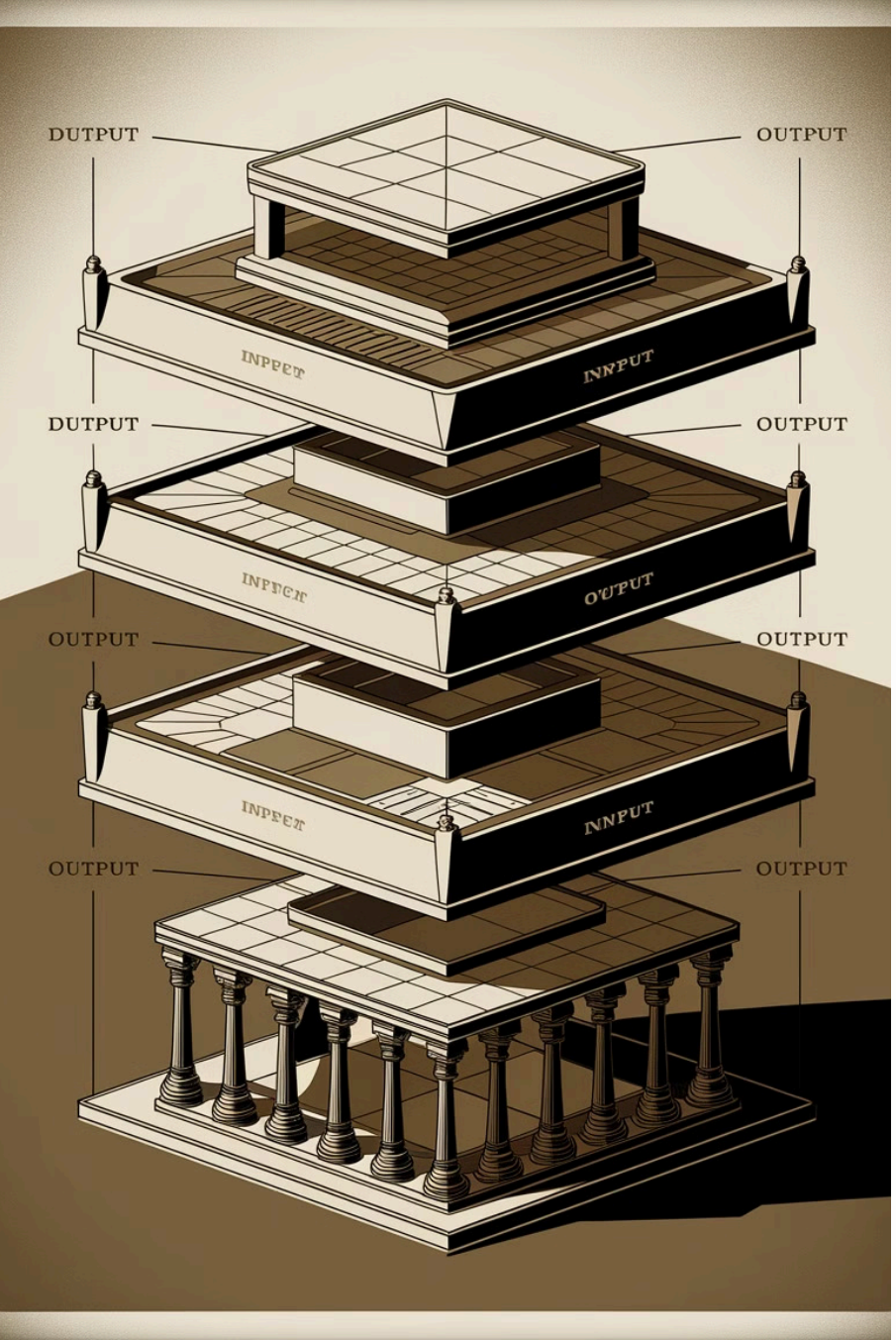
Layer Normalization

Stabilize learning

4

Residual Connections

Aid gradient flow



Transformer Architectures: Overview

Encoder-Only

For tasks like classification and QA

Decoder-Only

For text generation tasks

Encoder-Decoder

For tasks like translation and summarization

Encoder-Only: BERT

1 **Masked Language Modeling (MLM)**

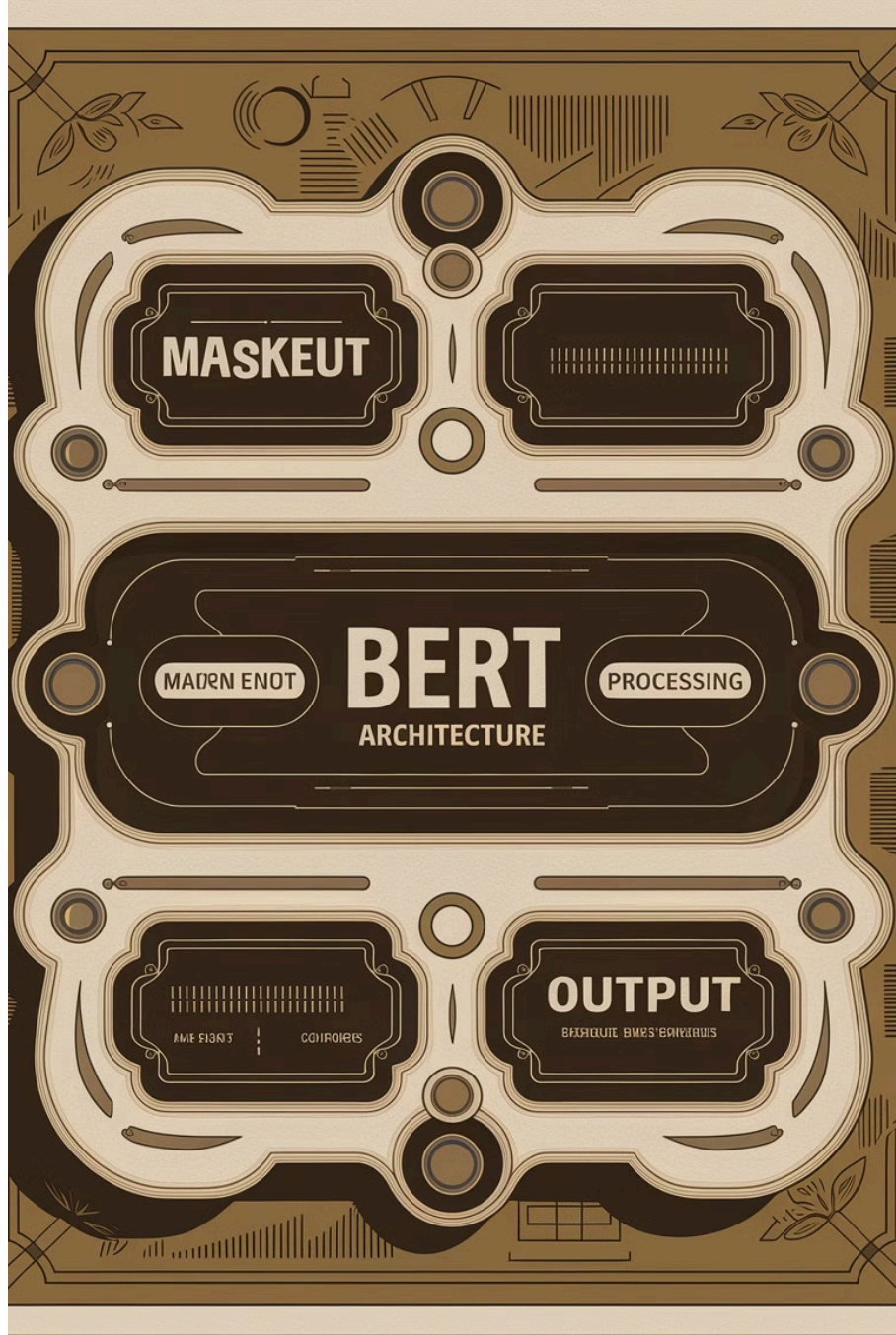
Predict masked tokens in input

2 **Next Sentence Prediction (NSP)**

Determine if two sentences are consecutive

3 **Applications**

Classification, Question Answering, etc.



Decoder-Only: GPT

1

Autoregressive LM

Predict next token based on previous ones

2

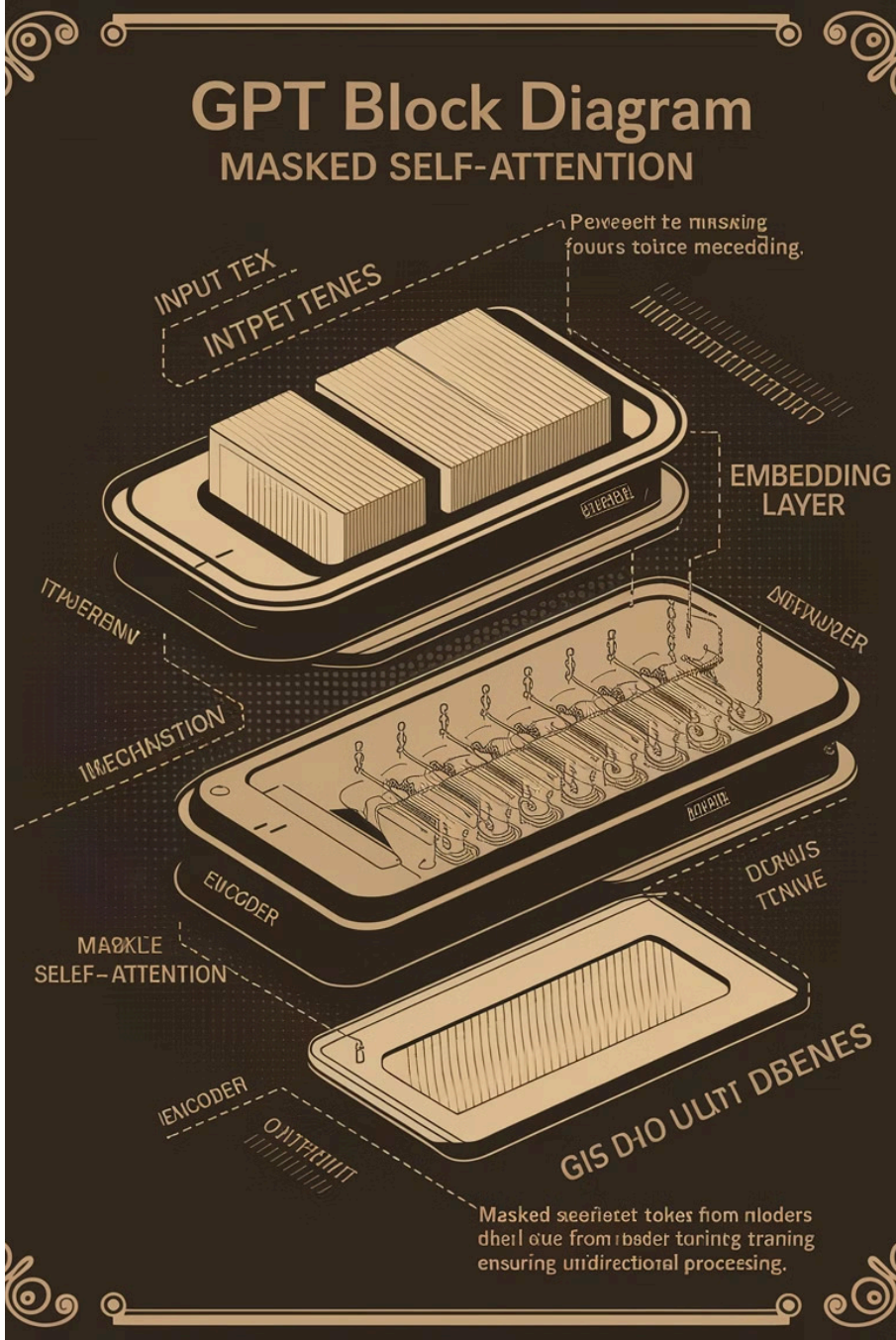
Causal Masking

Ensure one-directional information flow

3

Applications

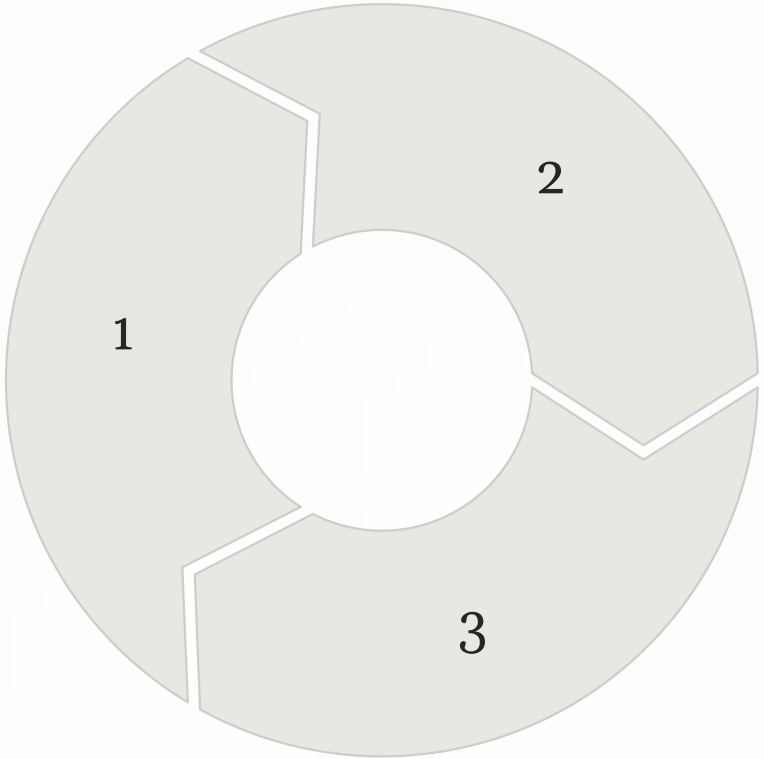
Text generation, chatbots, etc.



Encoder-Decoder: T5

Text-to-Text Framework

Versatile for various NLP tasks



Cross-Attention

Decoder attends to encoder outputs

Applications

Summarization, translation, etc.

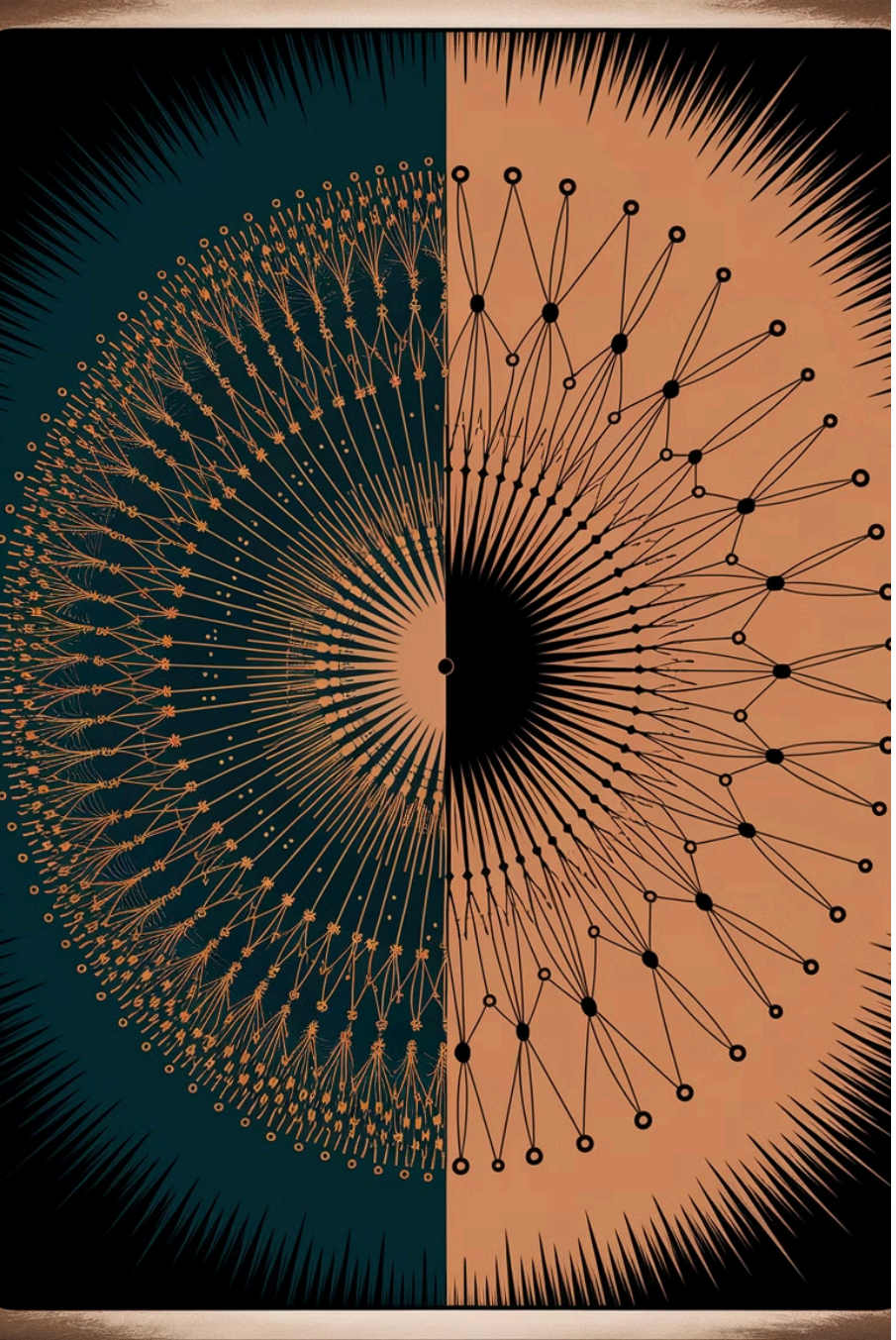
Cross-Attention Explanation

Encoder Outputs

Serve as Keys & Values for decoder

Decoder Processing

Uses its own Query to attend to encoder context



Recent Improvements: Efficient Attention

Flash Attention

Faster, memory-efficient kernel for attention computation

Sparse Attention

Reduces complexity by attending to subset of tokens (e.g., Big Bird, Longformer)

Advanced Topics: Cutting-Edge Research

Ring Attention

Efficient for extremely long sequences

RoPE in LLaMA

Advanced use of Rotary Positional Encoding

ALiBi Simplification

Incorporating relative biases efficiently

Conclusion and Future Directions

1

Efficiency

Ongoing research to reduce computational complexity

2

Scalability

Techniques for handling longer sequences

3

Adaptability

Transformers for diverse tasks beyond NLP